TEL200 Introduction to Robotics

Chapter 5: Navigation/Planning

David A. Anisi

Navigation/Planning

- Reactive (behavior-based) vs. Deliberate (map-based) planning

<u>Reactive (behavior-based)</u>

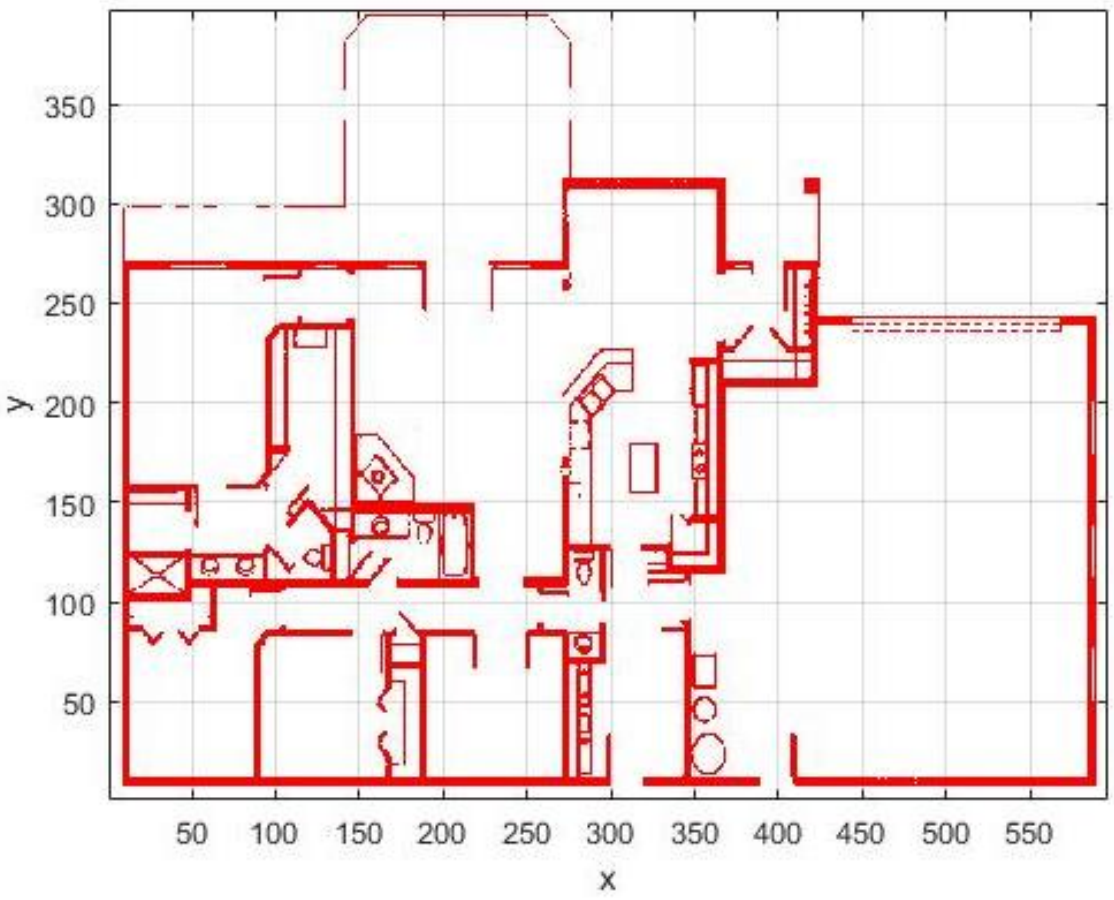
- Subsumption architecture
- Braitenberg vehicles
  - Phototaxis
  - Gas localization (infotaxis)
- Simple automata: bug algorithms

<u>Deliberate (map-based)</u>

- Mathematical world-representation
  - Occupancy grids/ Graphs/ Voronoi diagrams/ Octrees
- Dijkstra's, A*, D*
- Discrete Transform
- Roadmaps, PRM
- Dubins' / Reed-Shepp's / Lattice Planner
- RRT, RRT*

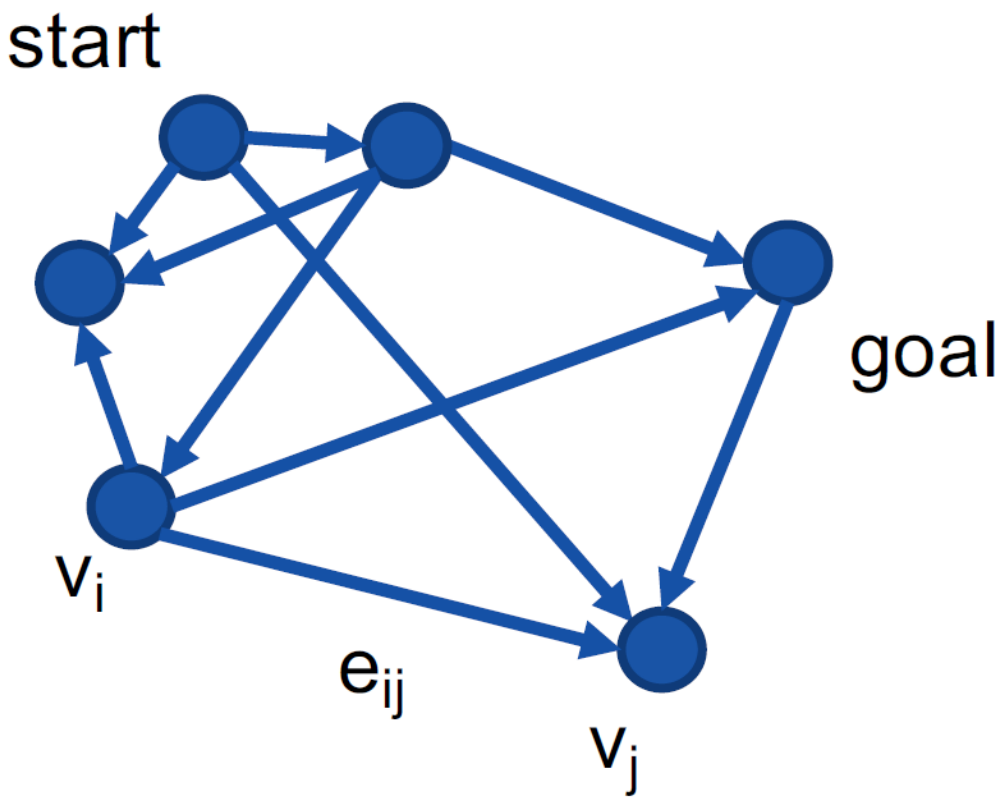
World-map representation: Occupancy grid

- (2D) Matrix with elements:
  - Obstacle = 1
  - Free space= 0
- Can be extended into a probabilistic setting where the value is in [0, 1]



Graphs – mathematical foundation for many planning algs.

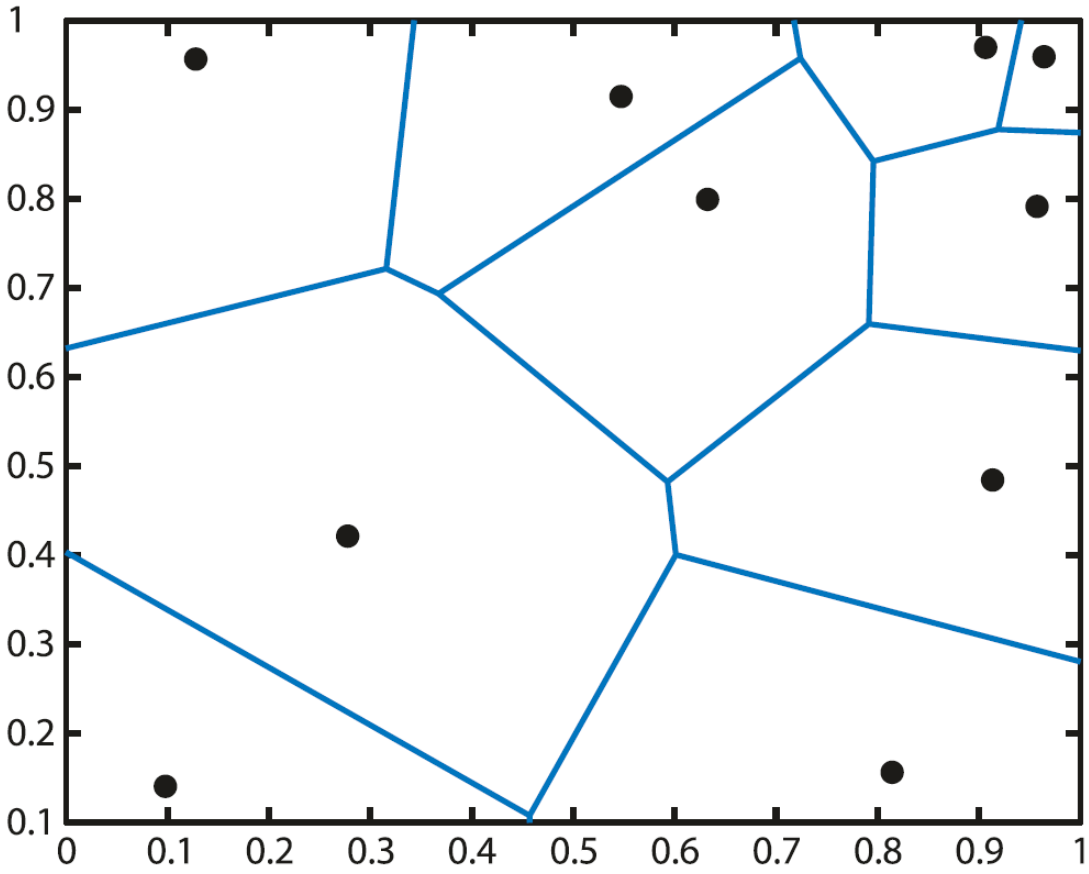
- Graph $G = (V, E)$
- Vertices/nodes: v_i
- Edges/links: $e_{ij} = (v_i, v_j)$
- Costs: c_{ij} (distance, risk, energy, …)
- Shortest path in graph found by Dijkstra's, A*, D*…



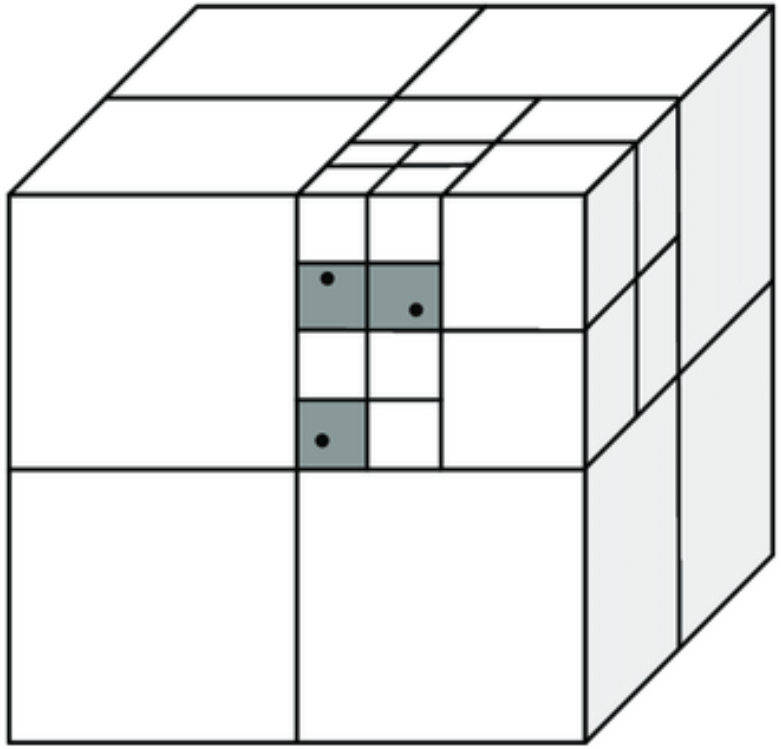
Voronoi diagram

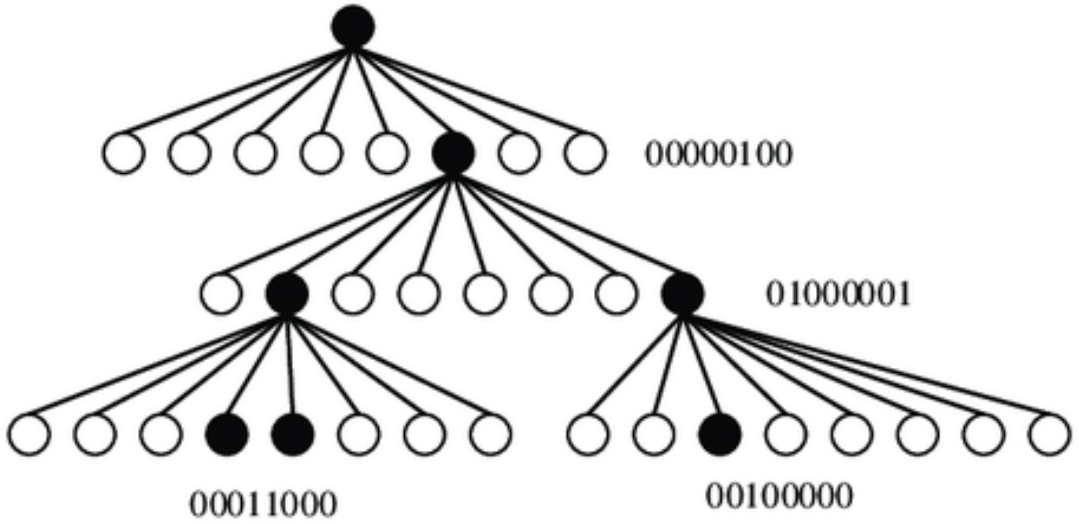
- Each cell corresponds to a site and consists of all points that are closer to its site than to any other site
- The edges of the cells are the points that are equidistant to the two nearest sites
- In Python:

```
>>> sites = np.random.uniform(size=(2,9));
>>> from scipy.spatial import Voronoi
>>> vor = Voronoi(sites.T);
```



Octrees – highly efficient and scalable





Serialized Octree:
00000100 01000001 00011000 00100000

Distance as cost

Distance

- Euclidean (L2-norm) distance

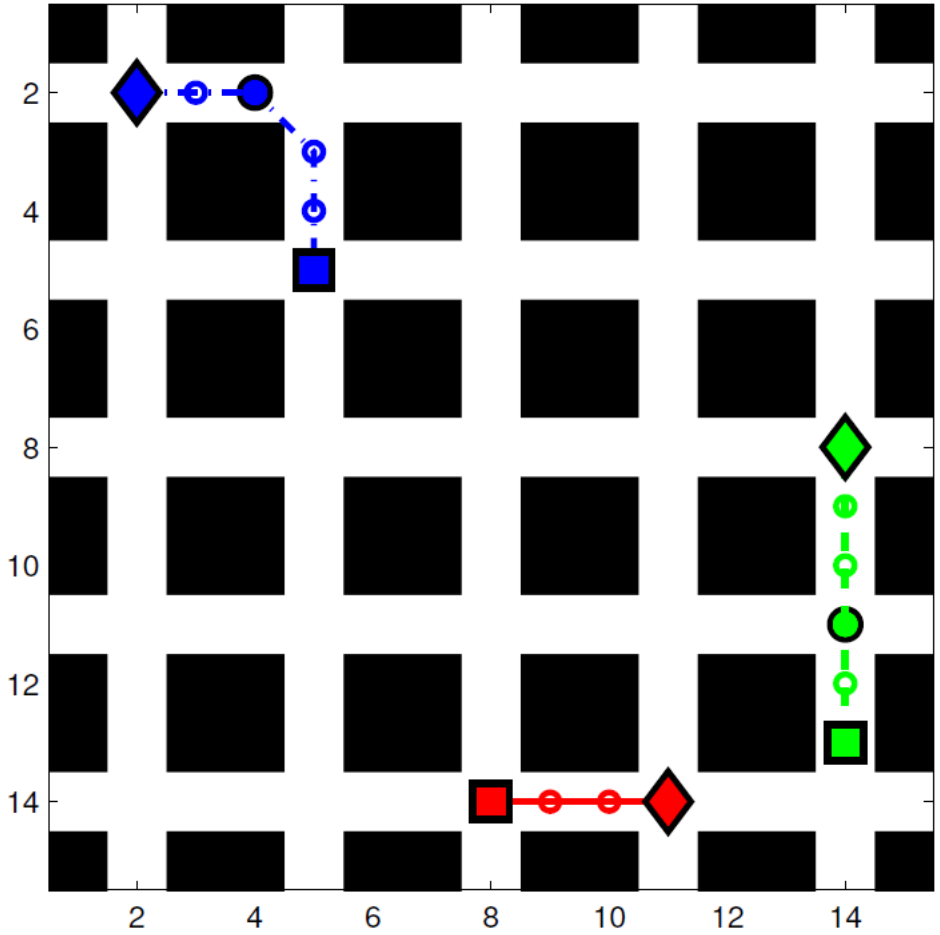
$$\|\boldsymbol{x}\|_2 := \sqrt{x_1^2 + \cdots + x_n^2}$$

- Manhattan (L1-norm) distance

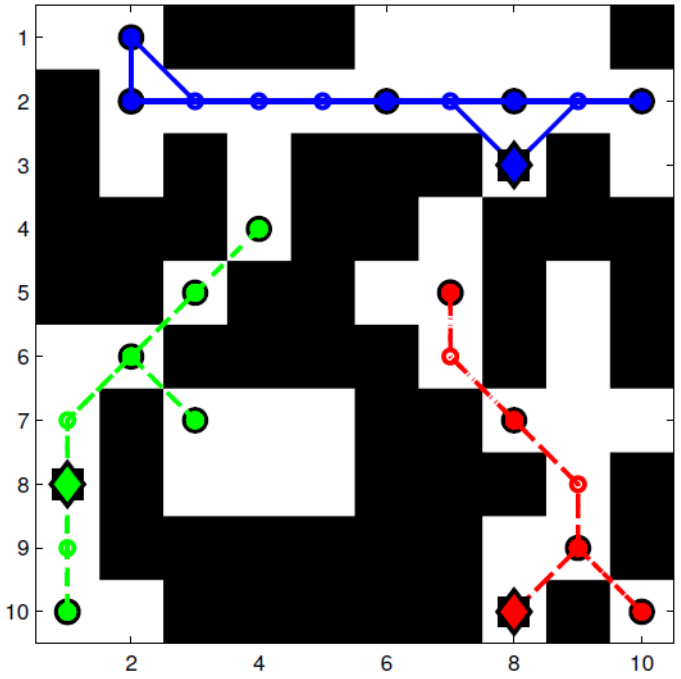
$$\|\boldsymbol{x}\|_1 := \sum_{i=1}^{n} |x_i|$$

- Max/L-infinity norm

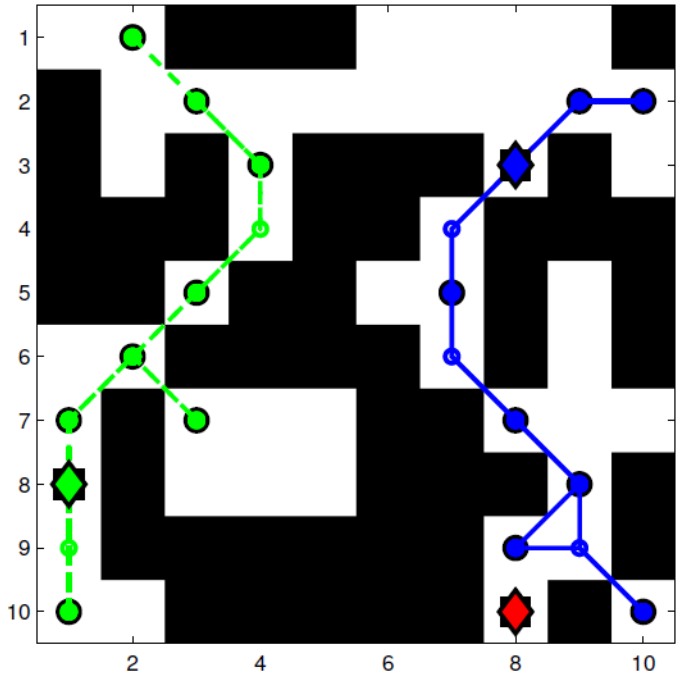
$$\|\mathbf{x}\|_\infty := \max_i |x_i|$$



Minimum time vs. Minimum cost

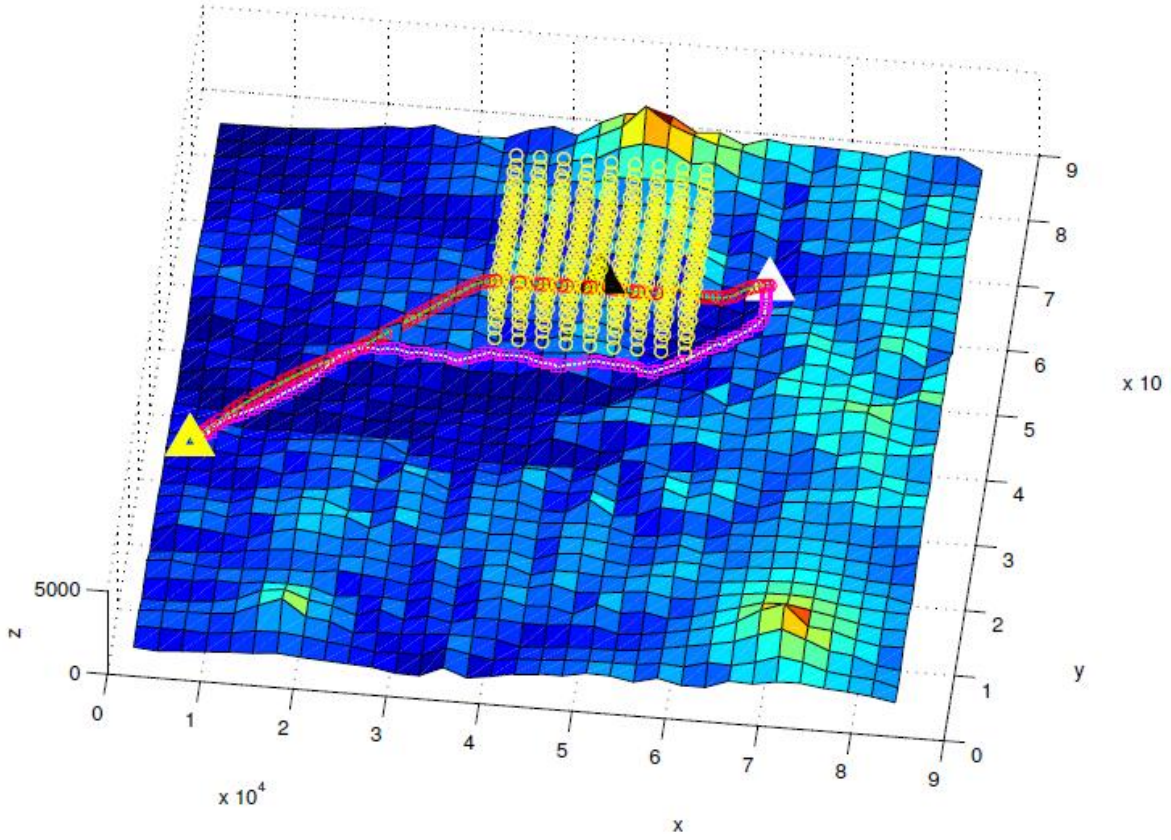


(a) Minimum time objective.



(b) Minimum distance objective.

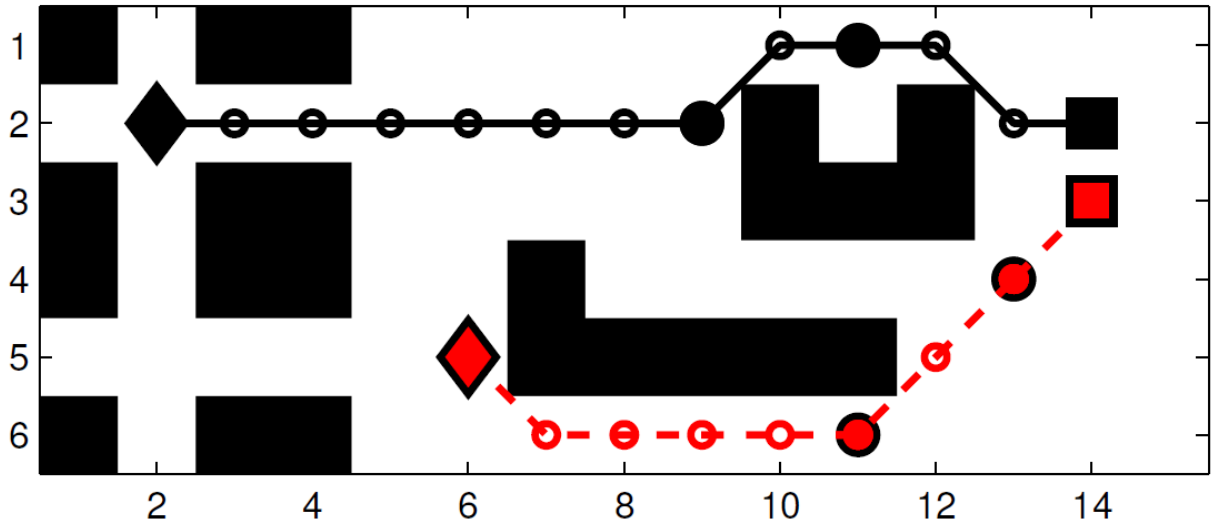
Minimum time vs. Minimum cost



Occupancy grid – 5 min with Cyrill

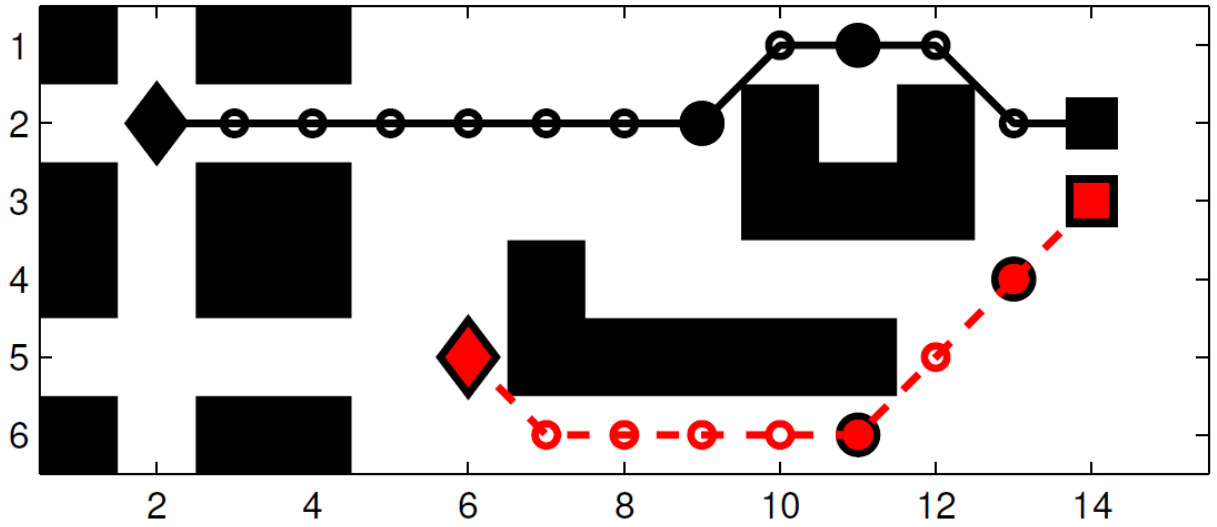
Map-based planning

- Assumptions on the robot:
  - Operate in a static grid world
  - Occupies one grid cell
  - Omnidirectional
  - Able to self-localize
  - Has knowledge about a global goal (position and orientation/direction)



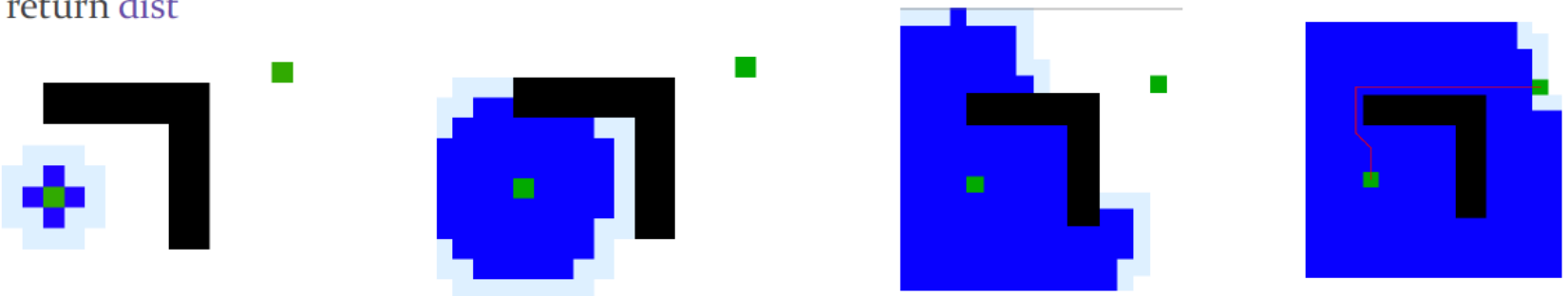
Map-based planning

- Two distinct phases:
  1. Pre-processing (pre-planning)
  2. Query
- Balancing of computational load between these two phases is crucial
- Considerations:
  - Nr of queries, size of environment, environmental changes, variations in start and/or goal?



Dijkstra's Algorithm (dynamic programming)

```
dist[s] ←0                                  (distance to source vertex is zero)
for  all v ∈ V–{s}
      do  dist[v] ←∞                        (set all other distances to infinity)
C←∅                                         (C, the set of closed vertices is initially empty)
Q←V                                         (Q, the queue initially contains all vertices)
while Q ≠∅                                  (while the queue is not empty)
do   u ← argmin(Q,dist)                     (select the element of Q with the min. distance)
     C←C∪{u}                                (add u to list of closed vertices)
      for all v ∈ neighbors[u]
           do  if   dist[v] > dist[u] + w(u, v)      (if new shortest path found)
                    then     d[v] ←d[u] + w(u, v)    (set new value of shortest path)

                                            (if desired, add traceback code)
return dist
```

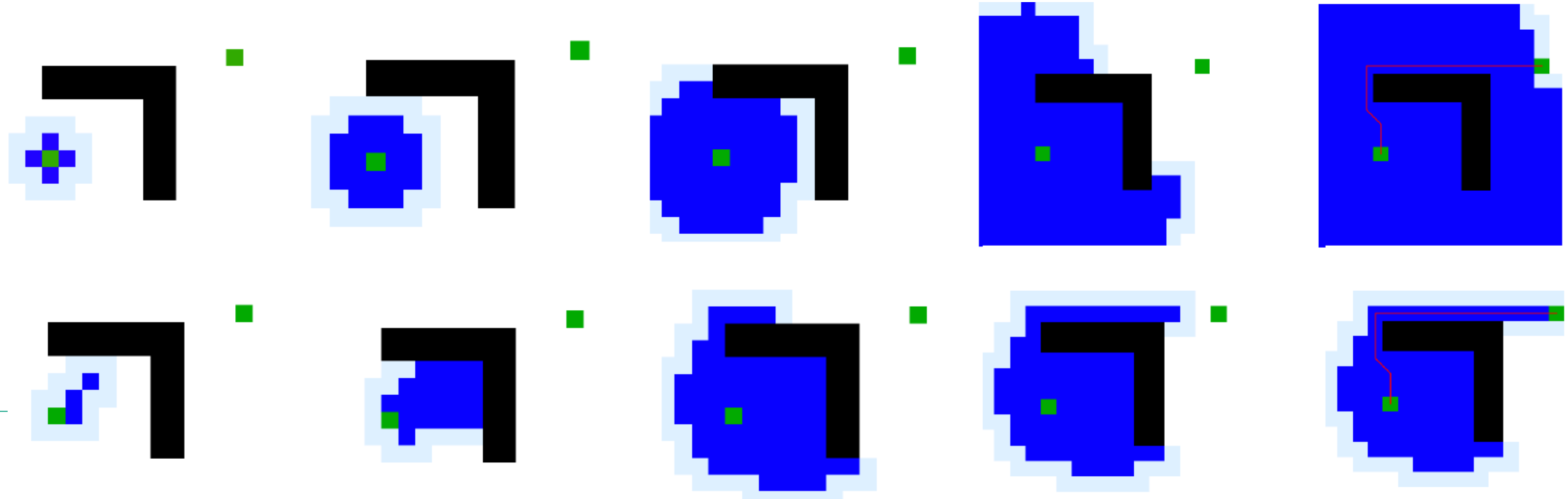


Dijkstra's vs. A*

The Closed set (C) grows

- As a circle in Dijkstra
- As an ellipsoid in A*

```
dist[s] ←0
for  all v ∈ V–{s}
      do  dist[v] ←∞
C←∅
Q←V
while Q ≠∅
do   u ← argmin(Q,dist + heur(u, start) )
     C←C∪{u}
      for all v ∈ neighbors[u]
            do  if   dist[v] > dist[u] + w(u, v)
                     then      d[v] ←d[u] + w(u, v)
```

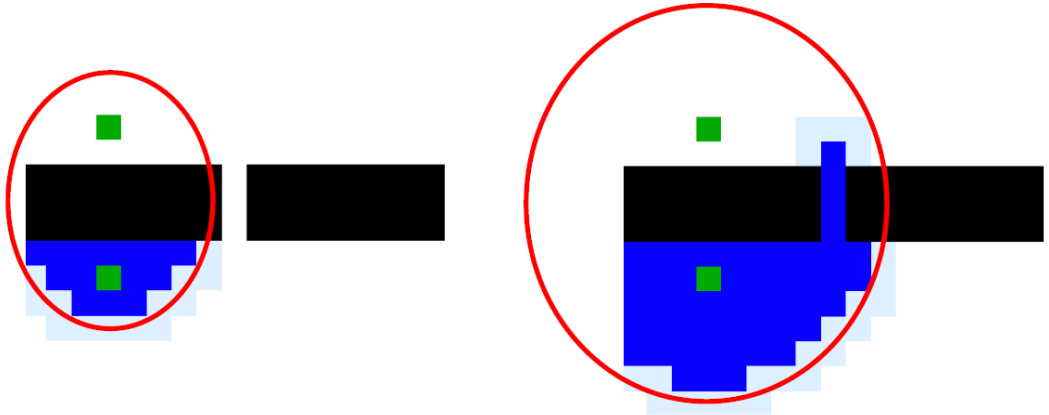


Dijkstra's vs. A*

The Closed set (C) grows

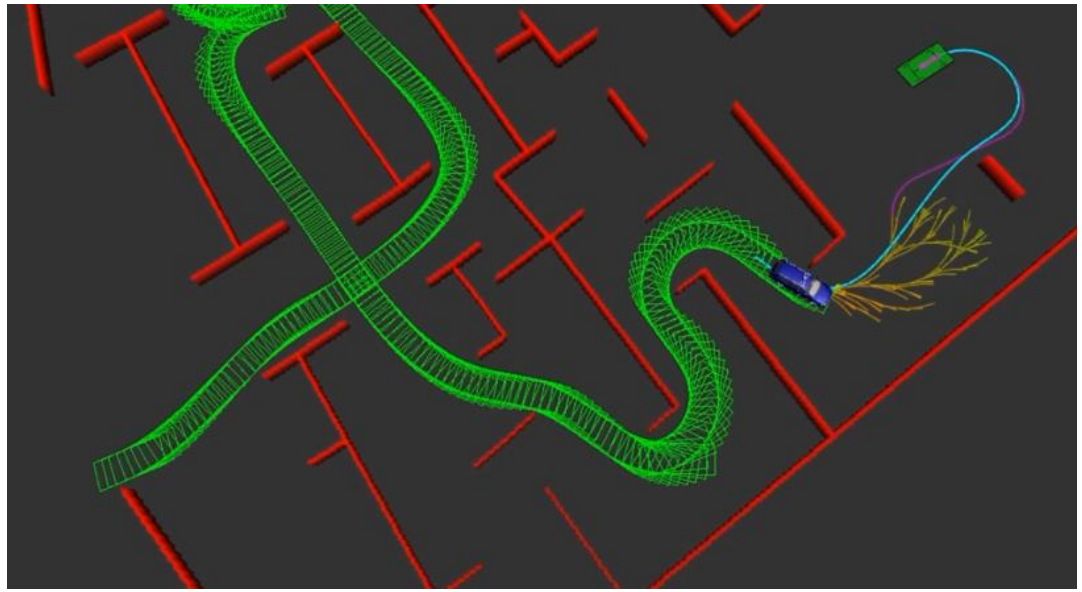
- As a circle in Dijkstra
- As an ellipsoid in A*

```
dist[s] ←0
for  all v ∈ V–{s}
      do  dist[v] ←∞
C←∅
Q←V
while Q ≠∅
do   u ← argmin(Q,dist + heur(u, start) )
     C←C∪{u}
      for all v ∈ neighbors[u]
            do  if   dist[v] > dist[u] + w(u, v)
                     then      d[v] ←d[u] + w(u, v)
```

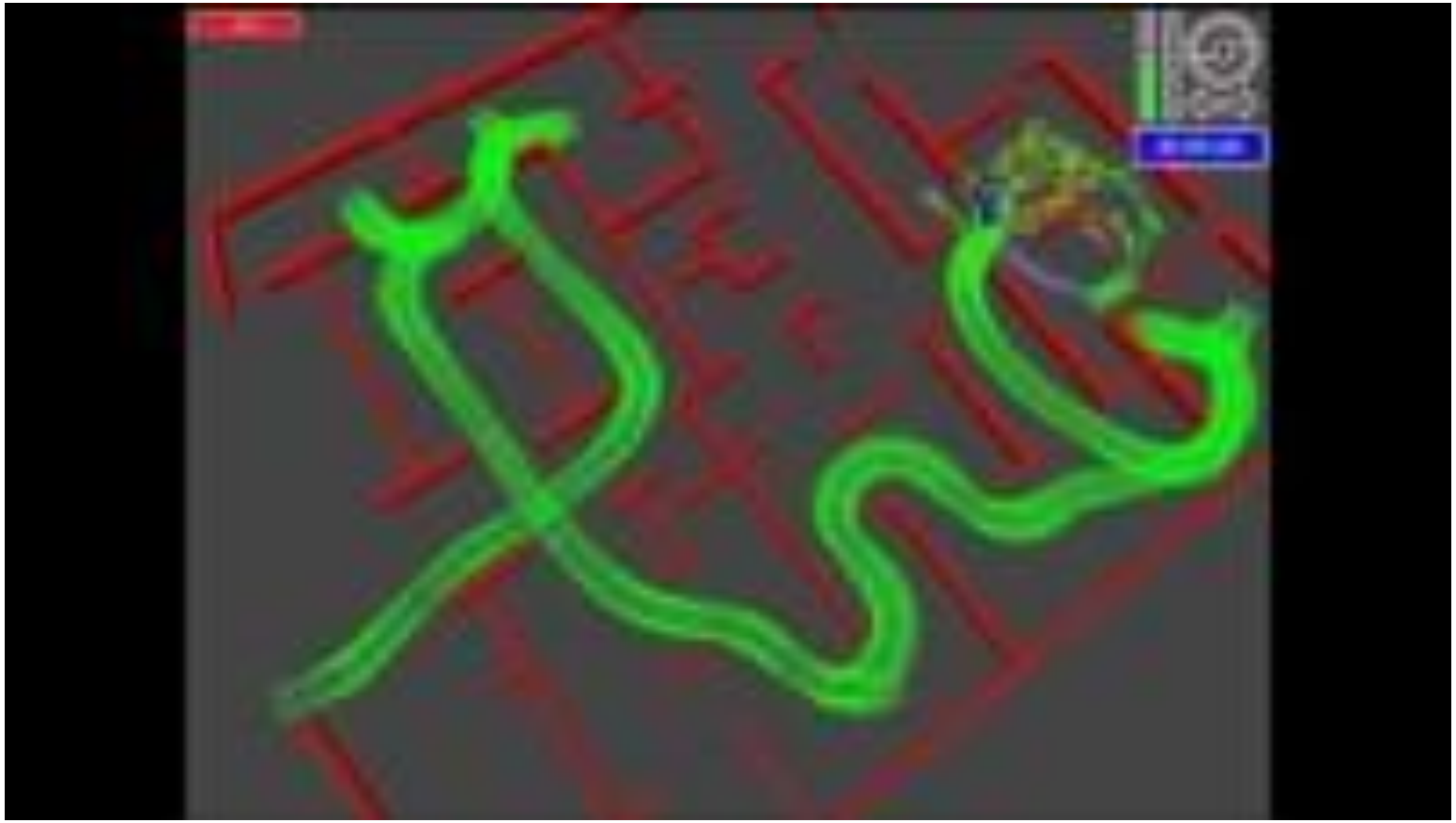


Extentions to consider

- Robot is not a point
- Robot does not live in R^2 (manipulator, drone,...)
- Motion is restricted by non-holonomic constraints (car-like, drone, ...)



A* in action – autonomus vehicle



Map-based planning: Discrete Transform

Given a goal position, the Distance Matrix has elements with the distance to the goal.

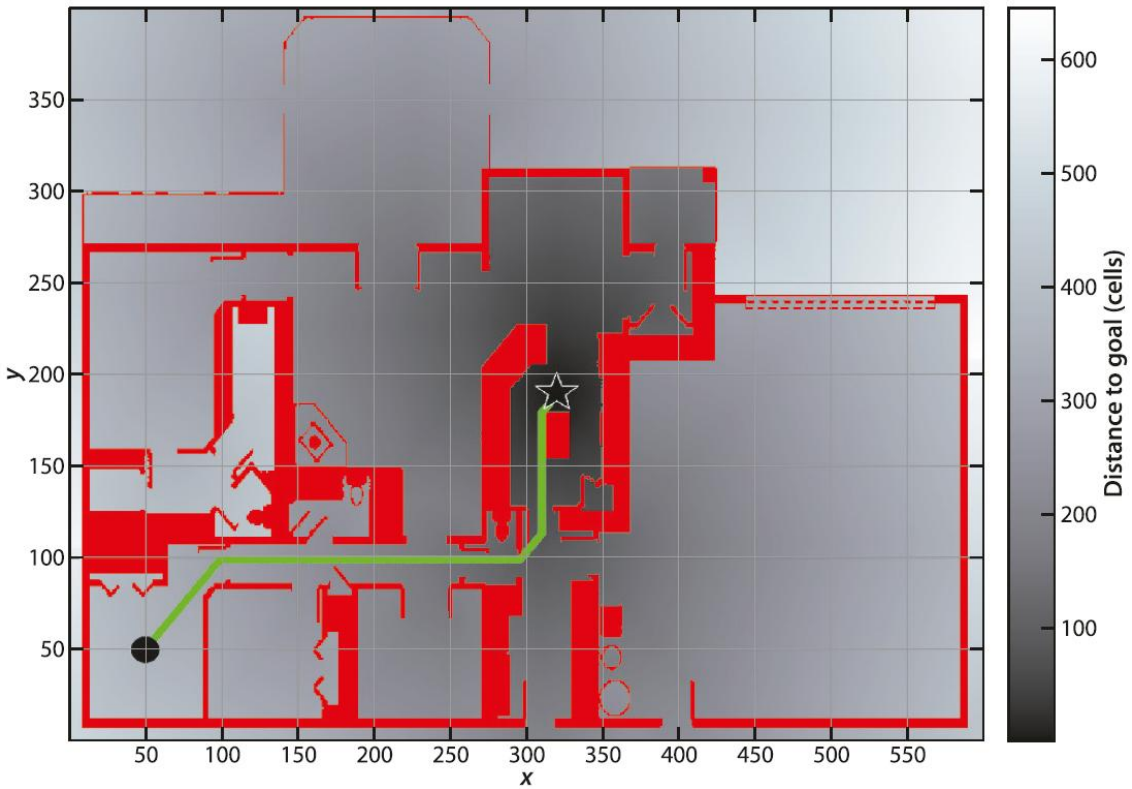
The distance must be feasible (circumvent obstacles).

Plan

```
>>> house = rtb_load_matfile("data/house.mat");
>>> floorplan = house["floorplan"];
>>> places = house["places"];
>>> dx = DistanceTransformPlanner(occgrid=floorplan);
>>> dx.plan(goal=places.kitchen)
>>> dx.plot();
```

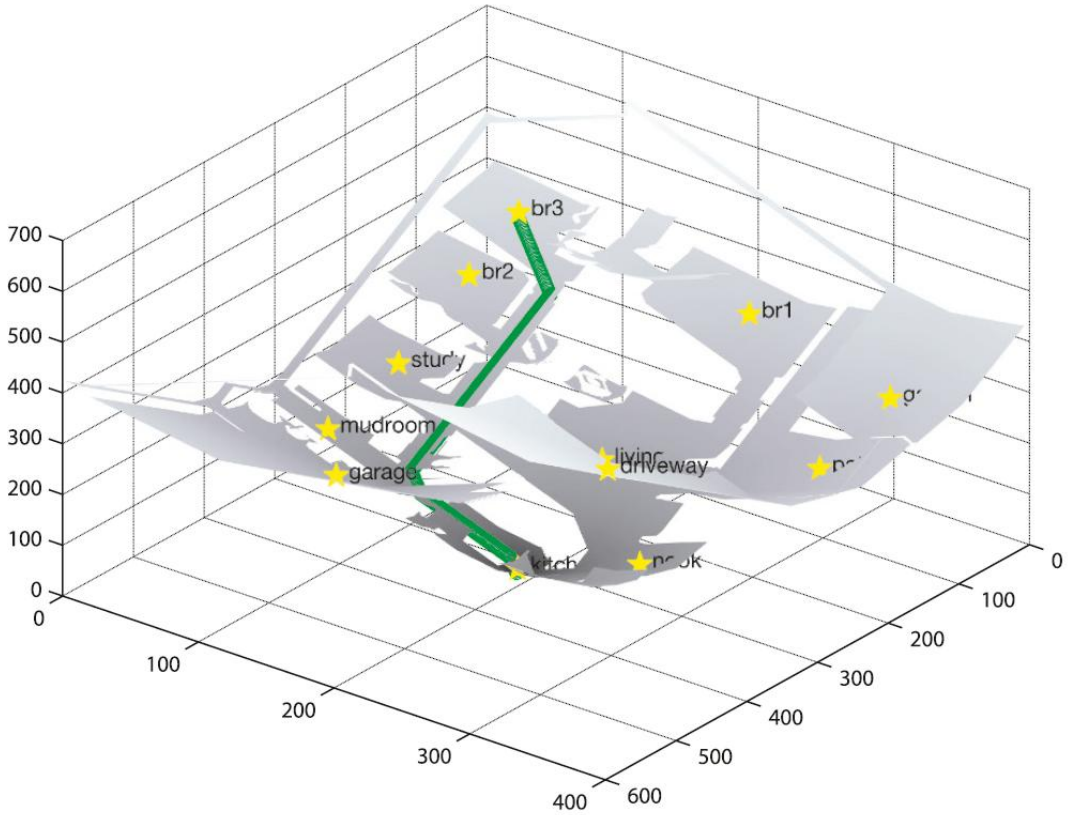
Query

```
>>> path = dx.query(start=places.br3);
>>> dx.plot(path);
```



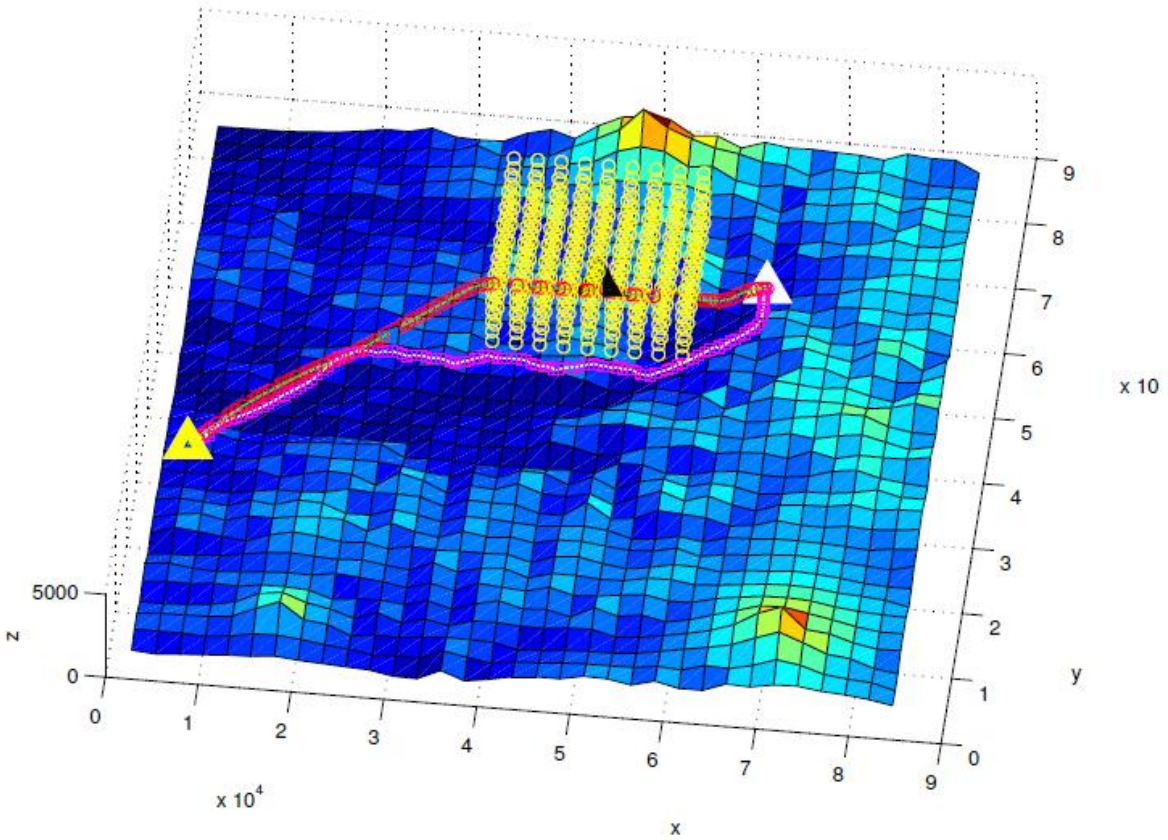
Map-based planning: Discrete Transform

- Provides optimal paths but expensive to calculate distance matrix
- Once done, it is valid for all starting points
- Dependent on goal position!
- Scalability limitations as size of matrix grow
- Graph-, roadmap- and probabilistic based approaches provide alternatives
- Potential field methods (not covered)



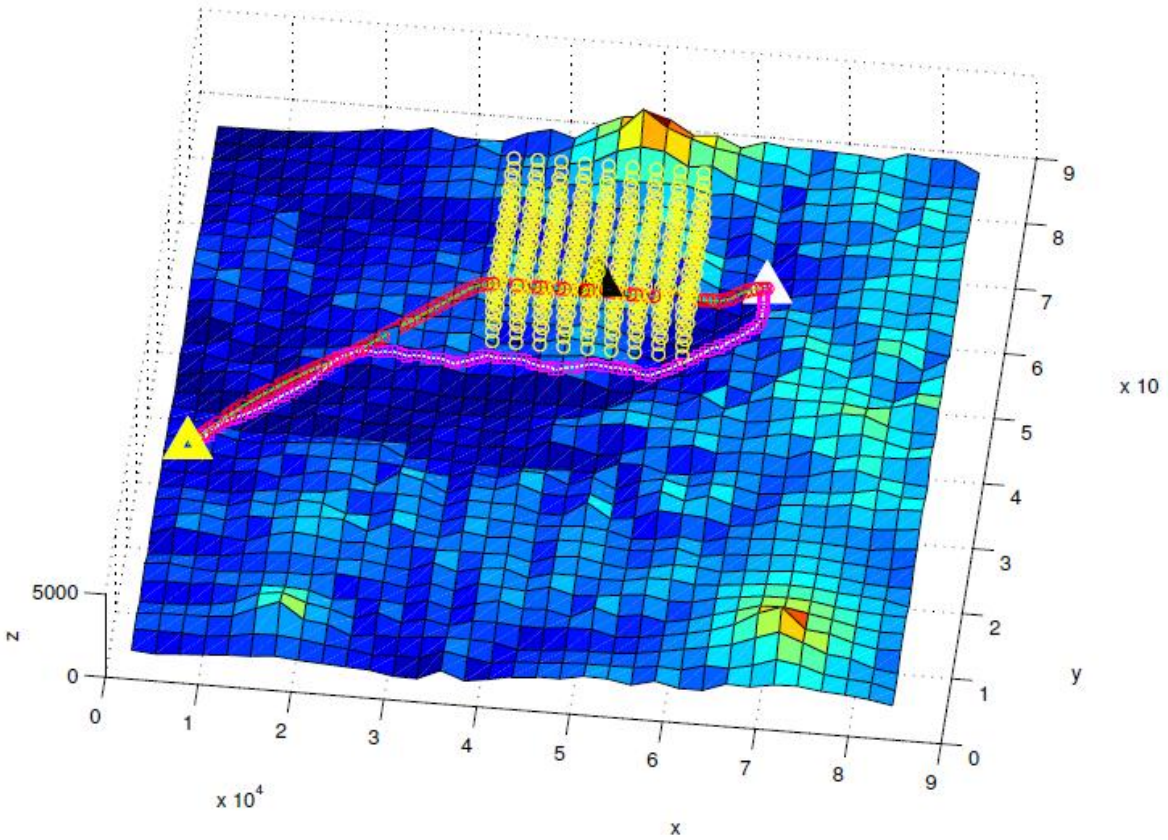
Map-based planning: D*

- D* (Dynamic A*) similar to A* except that the arc costs can change as the algorithm runs.
- A complete graph search algorithm specially designed for semi-structured (partially known) environments
- Introduced by Anthony Stentz 1994
- Used for planning by CMU DARPA Grand Challenge entry and Mars rovers



Map-based planning: D*

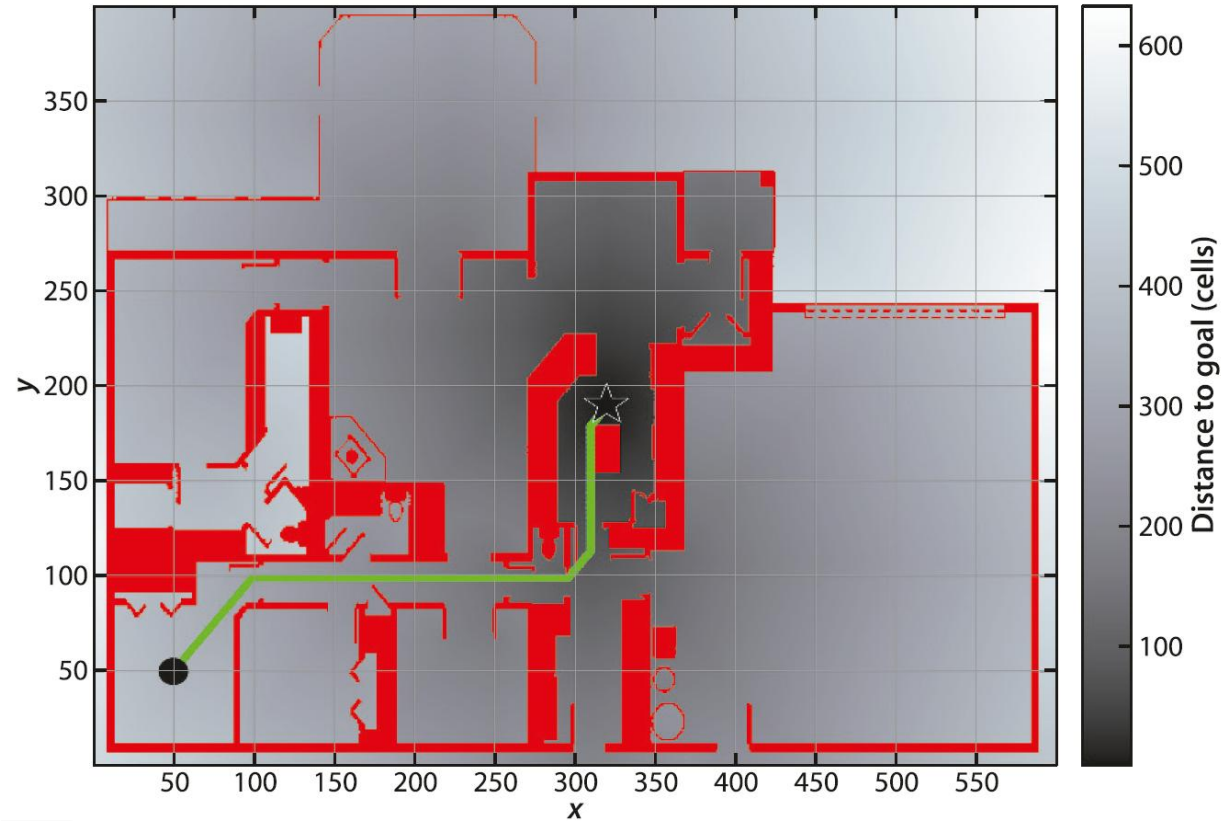
- Generalizes the cost notion as compared to e.g., occupancy grid
- Returns optimal paths
- Cheaper than DT, specially when re-planning incrementally
- Dependent on goal position!



D* in action

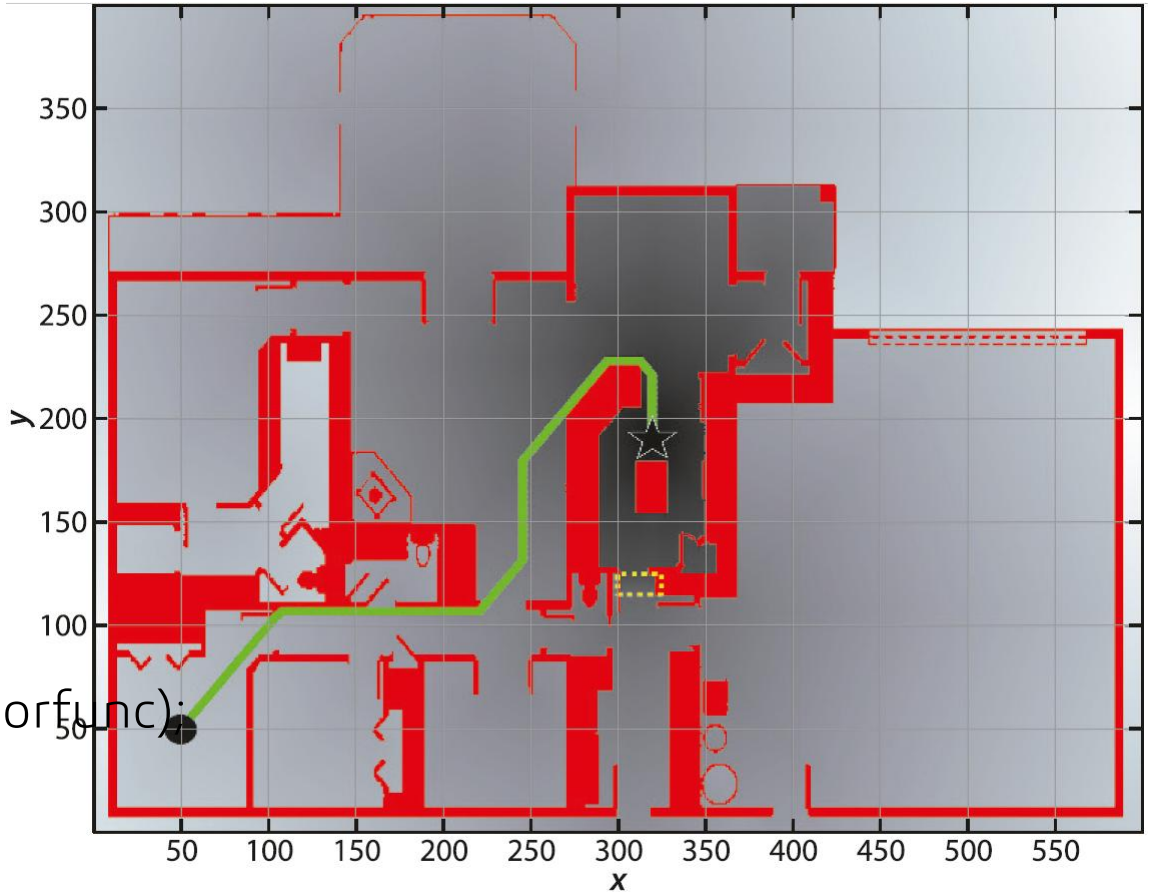
In Python;

```
>>> dstar =DstarPlanner(occgrid=floorplan);
>>> dstar.plan(goal=places.kitchen);
>>> nexpand0 = dstar.nexpand
359764
>>> path = dstar.query(start=places.br3);
```



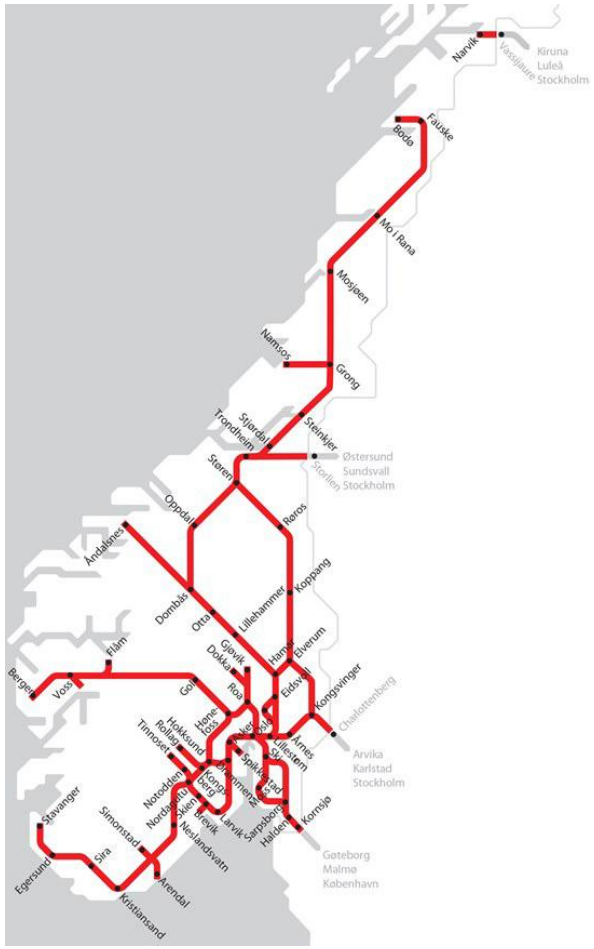
D* in action – map modification

```
>>> def sensorfunc(pos):
... if pos[0] == 300: # near the door?
... changes = []
... for x in range(300, 325):
... for y in range(115,125):
... changes.append((x, y, np.inf))
... return changes
>>>dstar.query(start=places.br3,sensor=sensorfunc)
>>> (dstar.nexpand - nexpand0) / nexpand0
0,1189
```



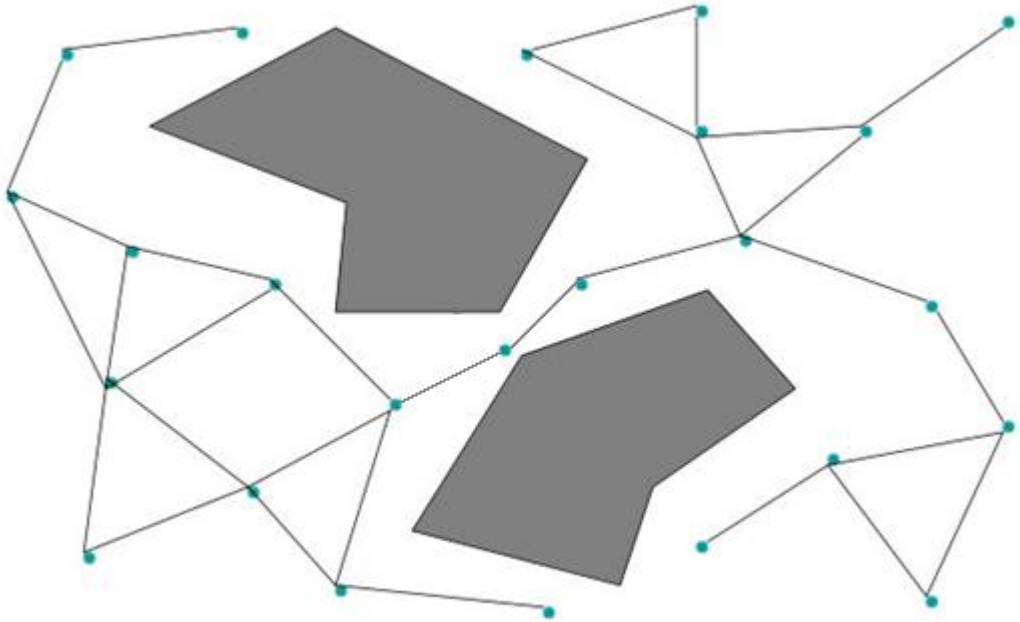
Roadmap based planning

- How can we make the expensive planning phase independent of start/goal configuration?
- Plan using a roadmap (cf. railway network)



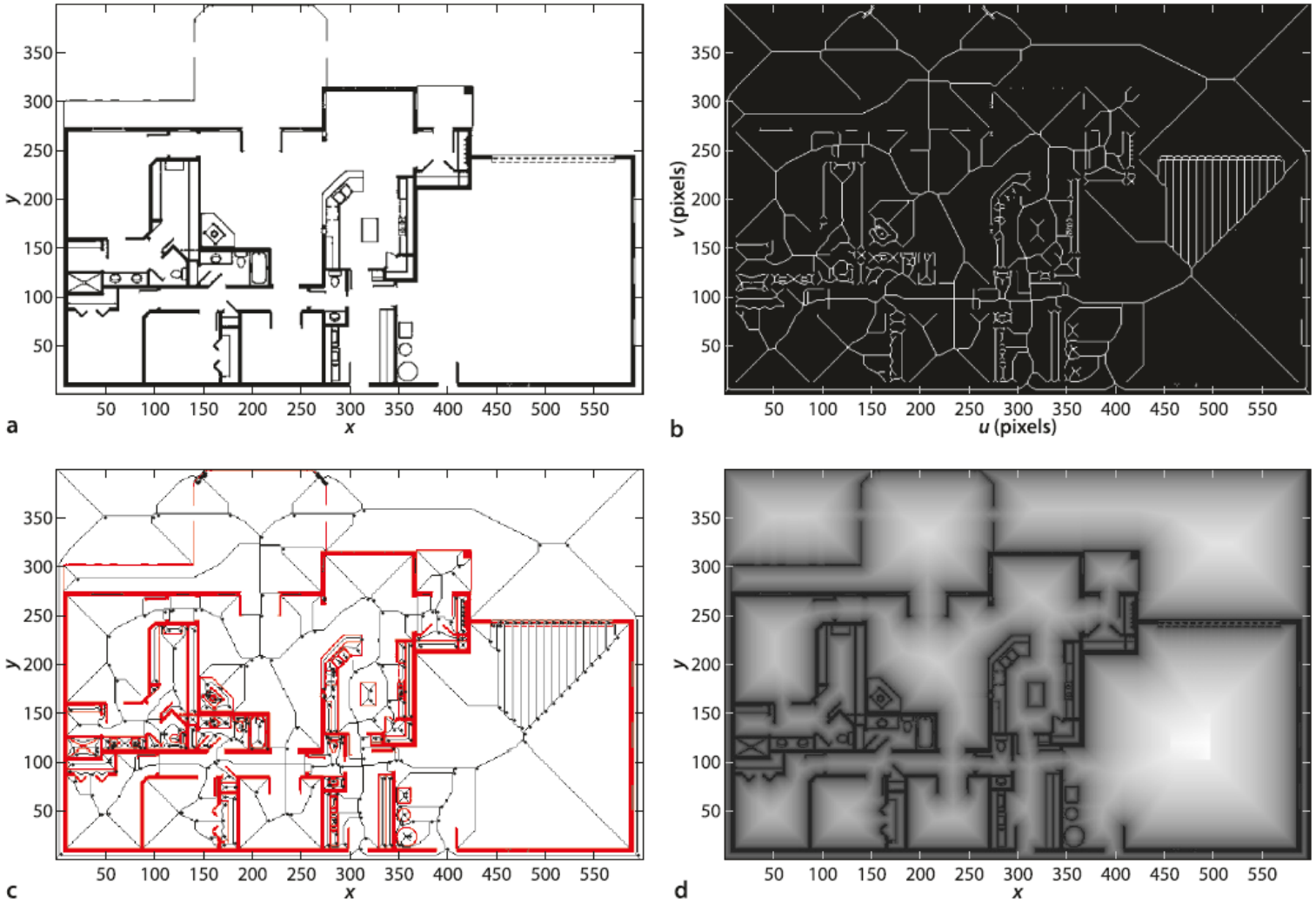
Roadmap based planning

- Connect start/goal to nearest nodes, n_s, n_g
- Calculate overall cost including shortest path between n_s, n_g
- The roadmap will be (semi)-static and function for arbitrary start/goal configuration



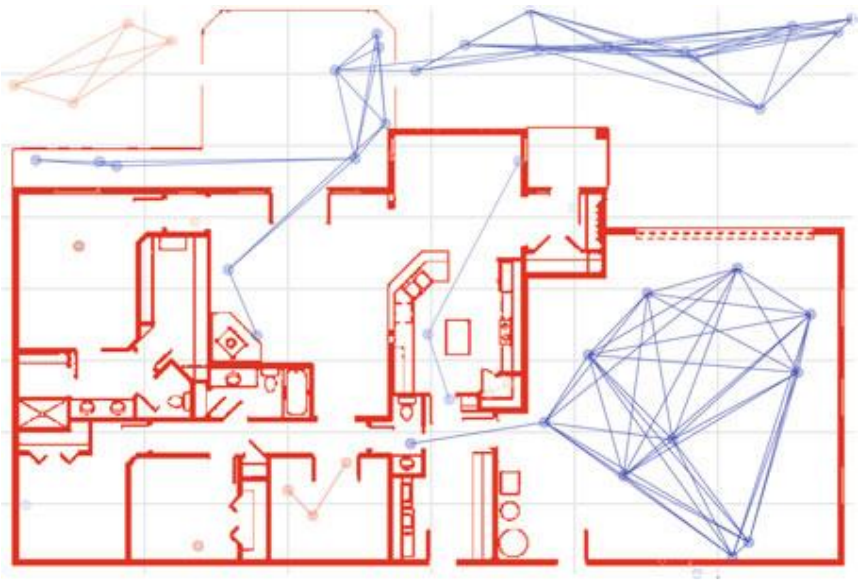
Voronoi Roadmap

- Free Space
- Topological skeleton
- Generalized Voronoi diagram
- Dist. transform of obstacles



Probabilistic Roadmap Method (PRM)

- Probabilistic methods sample the world-map (sparsely) and is therefore a more scalable iterative algorithm
- PRM random graph with 50 nodes
- Nb. Several disconnected subgraphs/ components

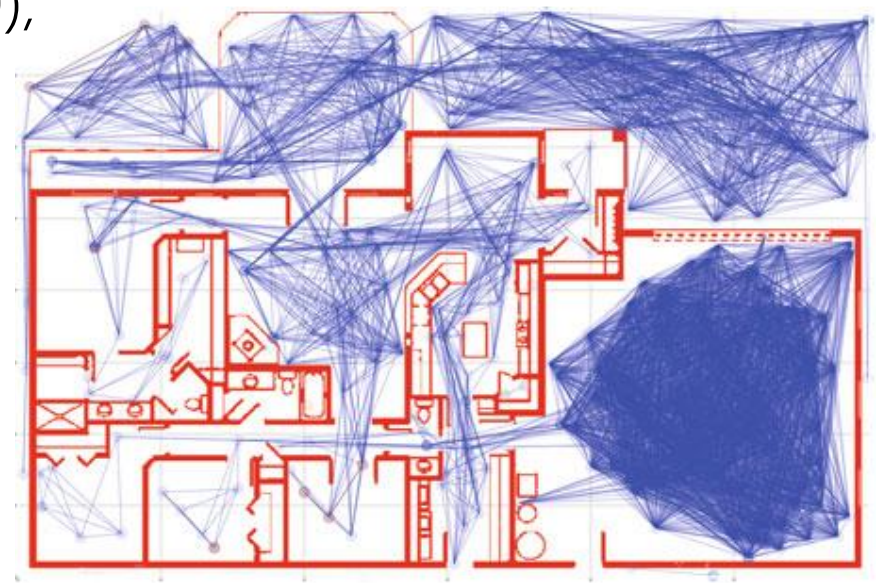


Probabilistic Roadmap Method (PRM)

Pre-processing phase in Python:

```
>>> prm = PRMPlanner(occgrid=floorplan, seed=0);
>>> prm.plan(npoints=300)
>>> prm.plot()
```

- Can calculate shortest path between any pair of nodes using Dijkstra's or A*
- Nb. not dependent on start/goal



Probabilistic Roadmap Method (PRM)

Query phase in Python:

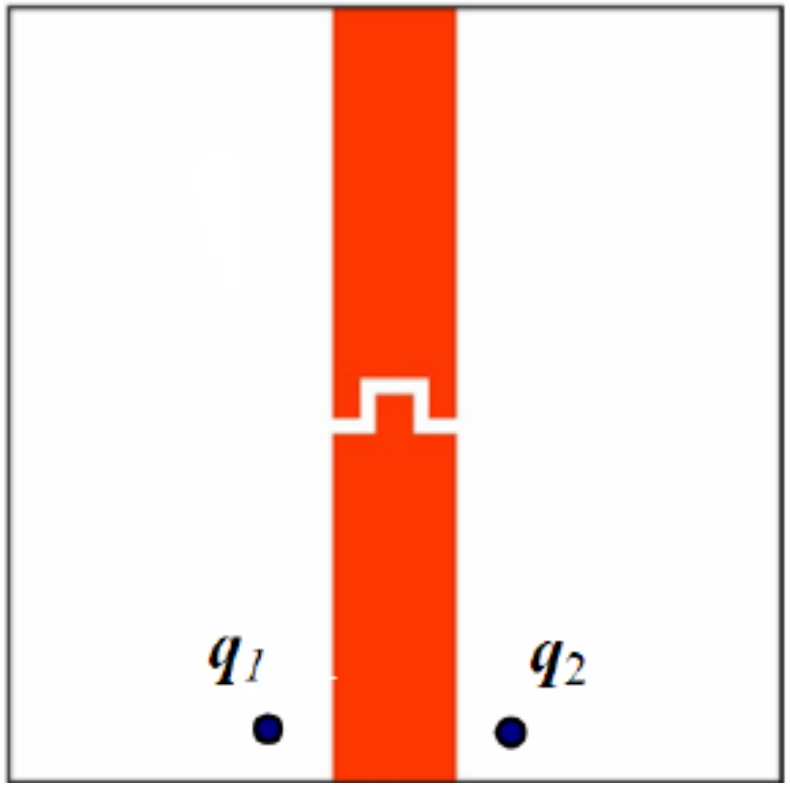
```
>>> path = prm.query(start=places.br2,
goal=places.kitchen);

>> prm.plot()
```

Probabilistic Roadmap Method (PRM)

Achieving this level of computational efficiency brings some trade-offs:

1. Different road-maps every time
2. May fail due to disconnected subgraphs
3. Is prone to adversarial examples

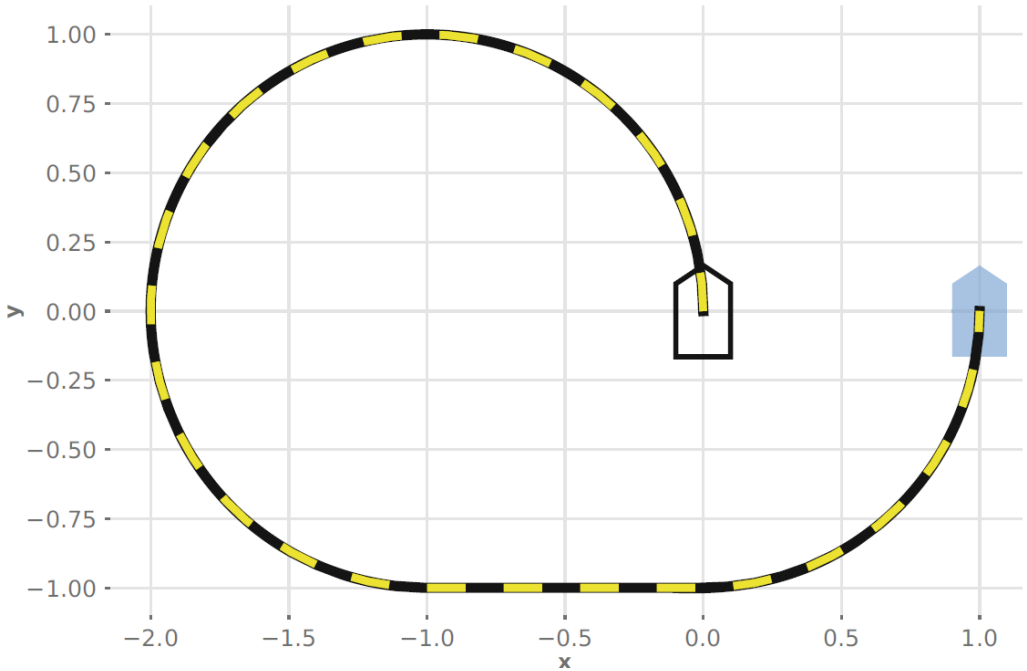
Student group activity: construct an environment where PRM is likely to fail



Driveable paths: Dubins'- and Reed-Shepp's car

- Planners considered so far may produce non-feasible paths not respeting the kino-dynamic (non-honomic) vehicle constraints
- Not able to follow them exactly
- Take motion (non-holonomic) constraints explicitly into account
- Dubins car:

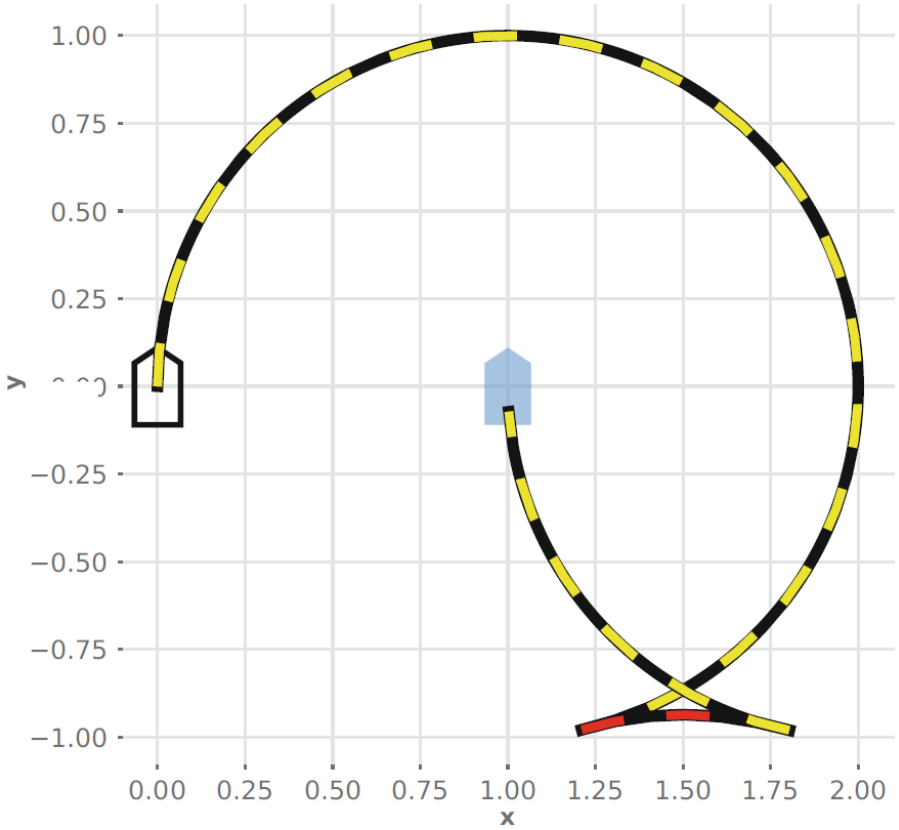
```
>>> dubins = DubinsPlanner(curvature=1)
```



Driveable paths: Dubins'- and Reed-Shepp's car

- Planners considered so far may produce non-feasible paths not respeting the kino-dynamic (non-honomic) vehicle constraints
- Not able to follow them exactly
- Take motion (non-holonomic) constraints explicitly into account
- Reed-Shepp's car:

```
>>> rs = ReedsSheppPlanner(curvature=1)
```



Driveable paths: Dubins'- and Reed-Shepp's car

- Optimal paths for unicycle model
- Dubins (v =1) and RS (v ∈{-1,1})
- Set of optimal paths: Concatenation of straight-line segments and circular arcs of maximum curvature (bang-bang solutions)

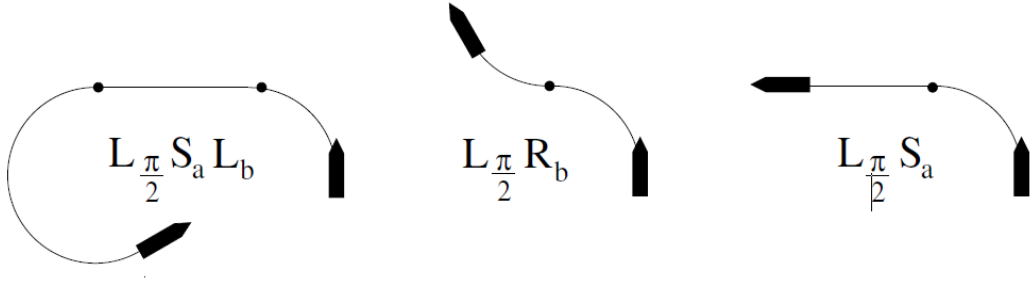


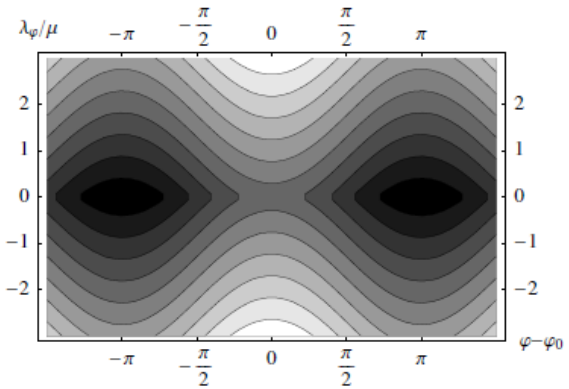
Figure 4.1: Examples of $L_{\frac{\pi}{2}} S_a B_b$ paths for Dubins' car.

FOI-R--0961--SE
July 2003
1650-1942

Scientific Report

David A. Anisi

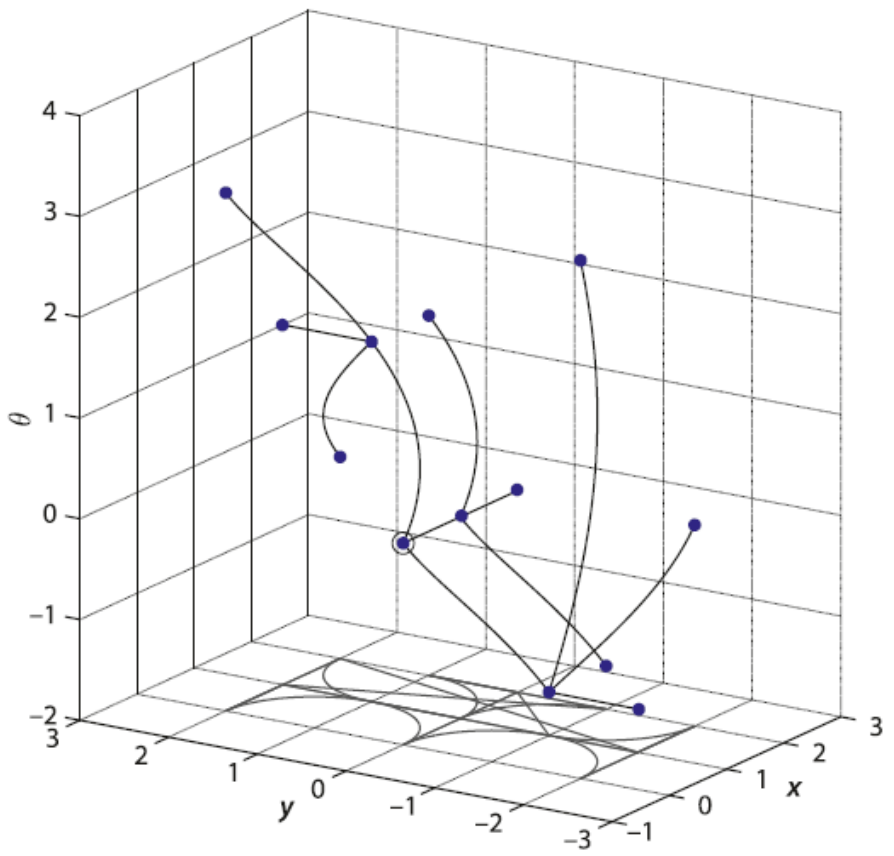
Optimal Motion Control of a Ground Vehicle



Driveable paths: Lattice Planner

- Take motion (non-holonomic) constraints explicitly into account
- Use «Dubins curve» with straigh lines and arcs with constant/max turning
- Pre-processing phase in Python:

```
>>> lp = LatticePlanner();
>>> lp.plan(iterations=1)
>>> lp.plot()
>> lp.plan('iterations', 8)
```



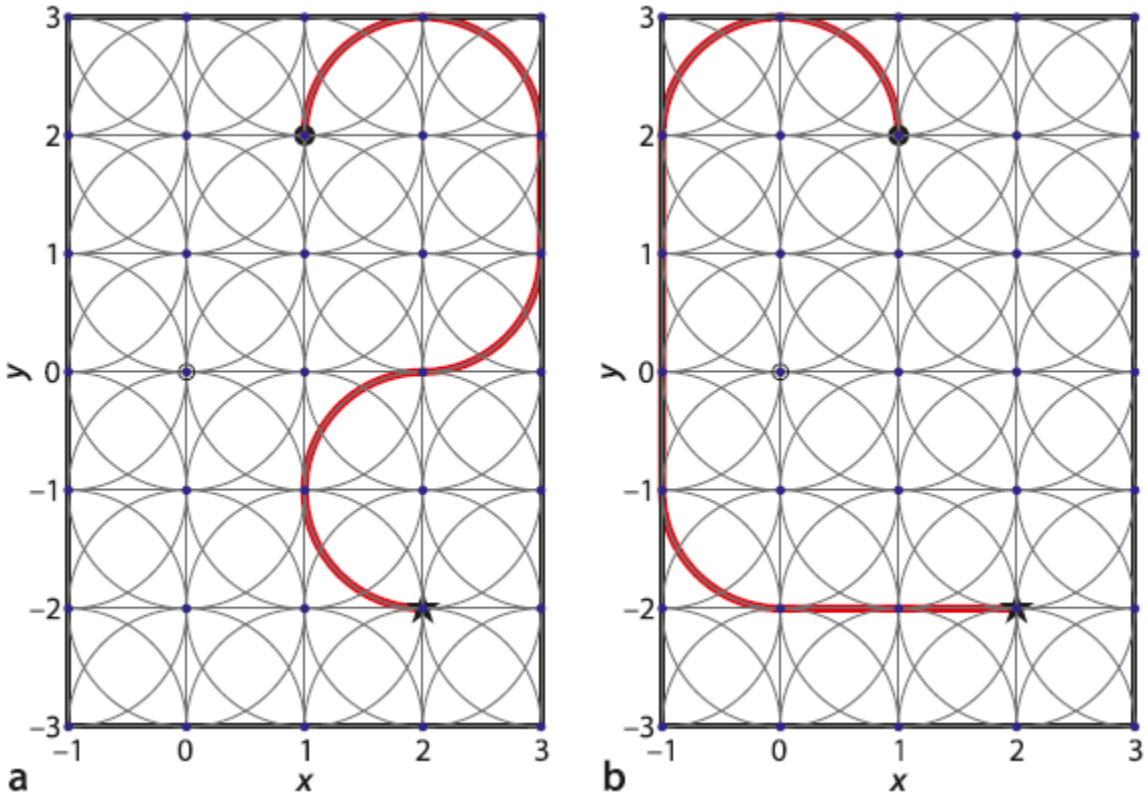
Driveable paths: Lattice Planner

- Query phase in Python:

```
>>> qs = (0, 0, pi/2); q =(1, 0, pi/2);
>>> path, status = lp.query(qs, qg);
>>> lp.plot(path)
```

- Penalizing turning more:

```
>>> lattice = LatticePlanner(costs=[1, 10, 1])
>>> lp.plan(iterations=8)
>>> path, status = lp.query(qs, qg)
```



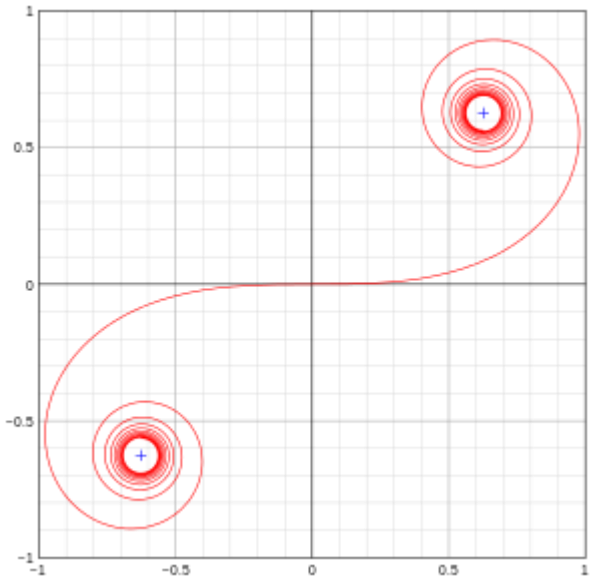
Driveable paths: Continuous curvature polynomials

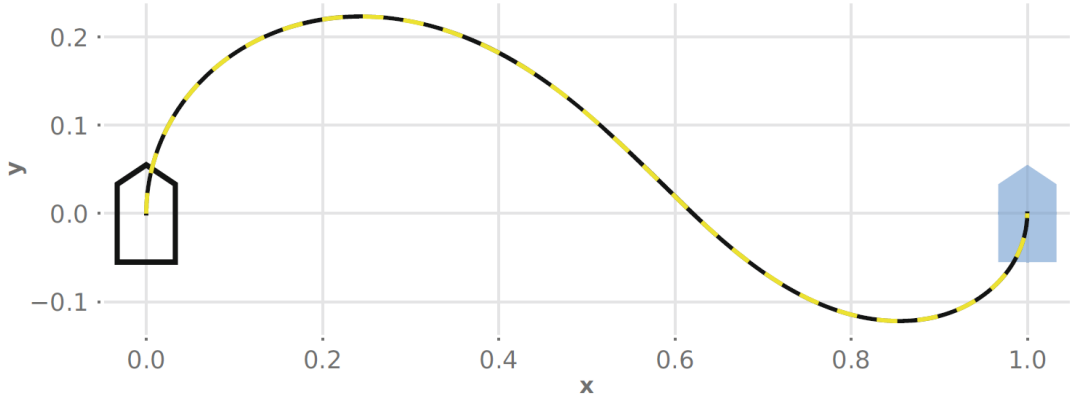
- These planners produce continuous (C), but not smooth (C^1, C^2) paths
- Curvature is discontinuous

Model name	Unicycle Robot Model	Car-like Model	Front-wheeled Car
System Dynamics	$\dot{x} = v\cos\varphi$ $\dot{y} = v\sin\varphi$ $\dot{\varphi} = \omega$	$\dot{x} = v\cos\varphi$ $\dot{y} = v\sin\varphi$ $\dot{\varphi} = \frac{v}{L}\tan\delta$	$\dot{x} = v\cos\varphi$ $\dot{y} = v\sin\varphi$ $\dot{\delta} = \omega$ $\dot{\varphi} = \frac{v}{L}\tan\delta$
Path Smoothness	C	C^1	C^2

Car-like robots: paths with continuous curvature

- **Clothoid**: curvature changes linearly with its curve length
- Roads are normally curved as clothoids

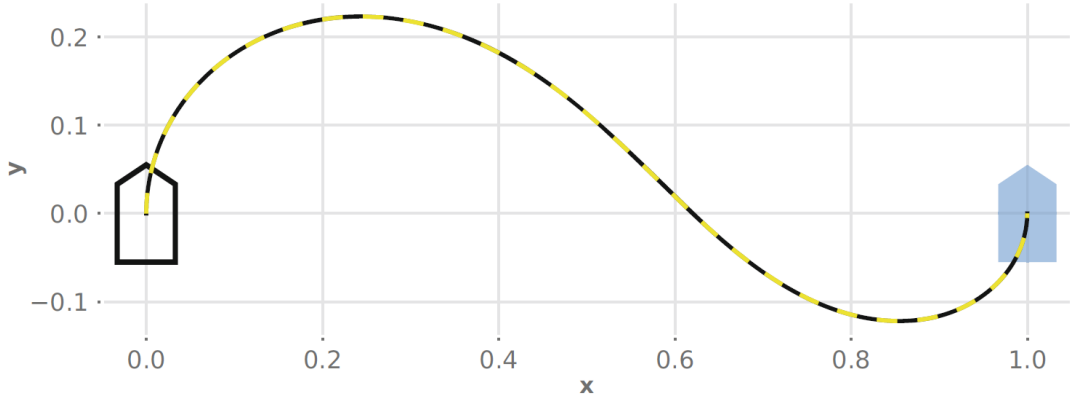




Car-like robots: paths with continuous curvature

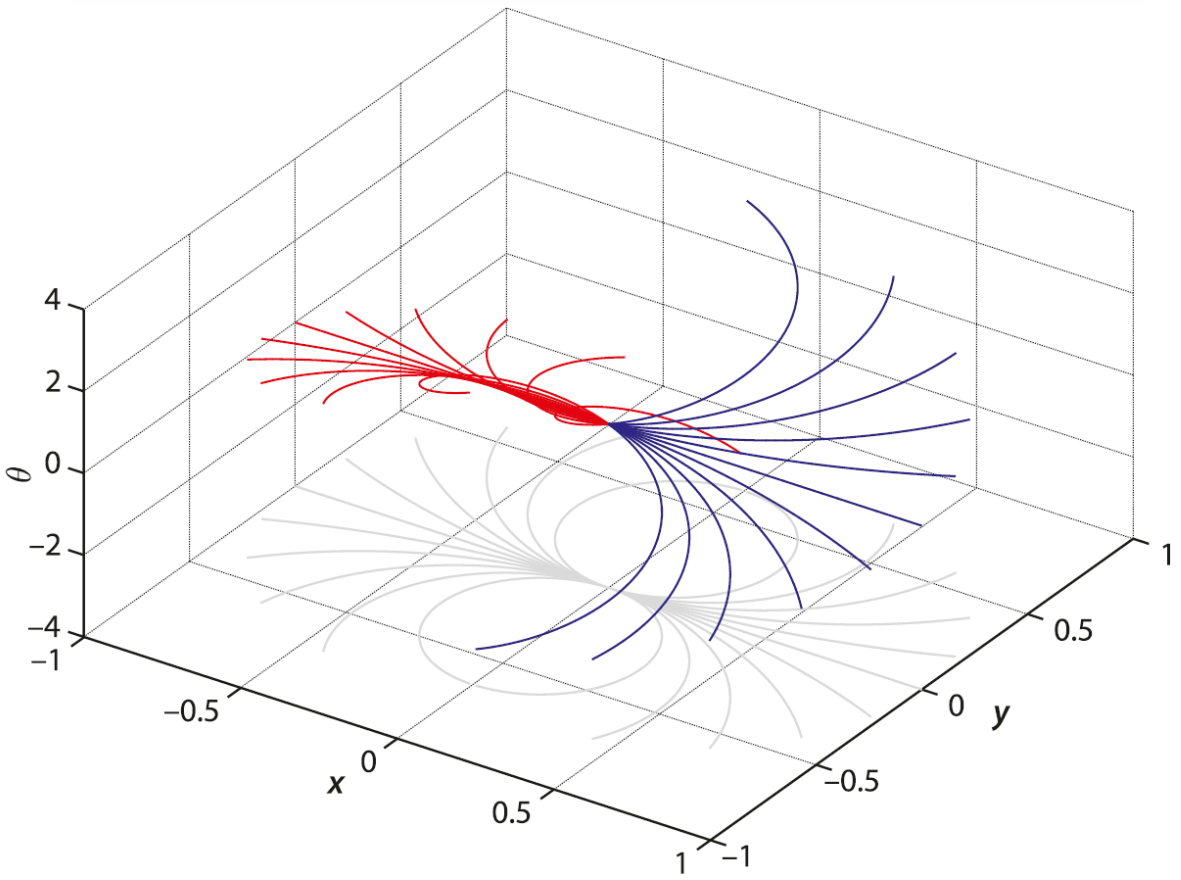
- **Clothoid**: curvature changes linearly with its curve length
- Roads are normally curved as clothoids

```
>>> cpoly = CurvaturePolyPlanner()
>>> path, status = cpoly.query(qs, qg)
>>> status
CurvaturePolyStatus(length=1.299,
maxcurvature=9.9549,
poly=array([ 3.059, 3.095, 2.234, -
4.865]))
>>> cpoly.plot(path);
```



Rapidly-Exploring Random Tree (RRT)

- Sampling-based method (like PRM)
- Able to take the motion model into account (like Latice planner).



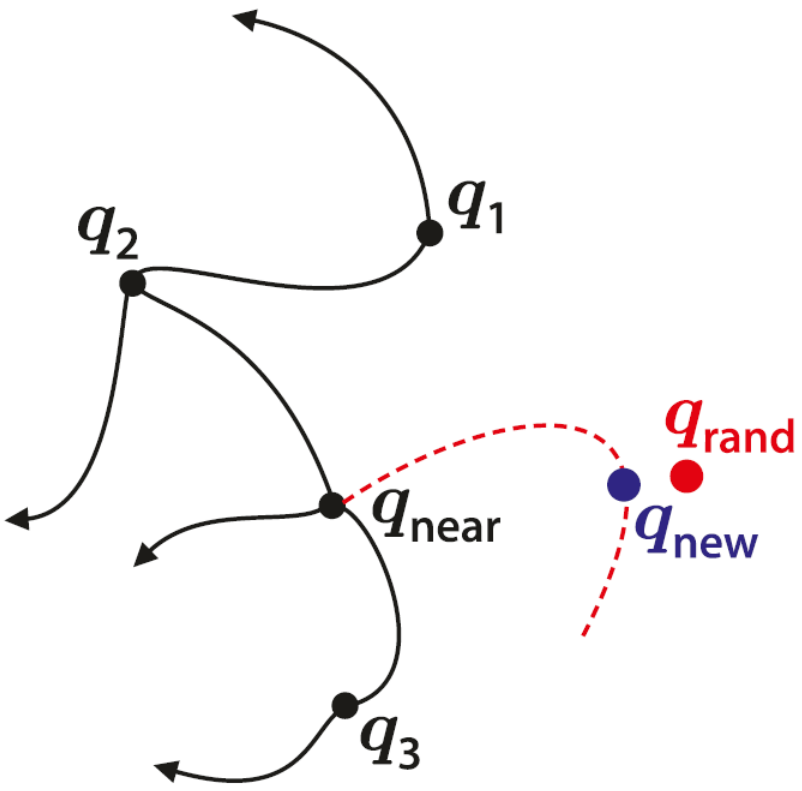
(C) Javad Hossein

Rapidly-Exploring Random Tree (RRT)

Main steps:

- Given a graph of robot configurations with vertices $V = \{q_1, q_2, \dots\}$, $q_i \in \mathbb{R}^2 \times \mathbb{S}^1$

1. Generate a new random config, q_{rand}
2. Find closest* config, q_{near}
3. Compute feasible control that moves robot from q_{near} towards q_{rand}
4. If no collision, add closest config q_{new} to graph. Goto1



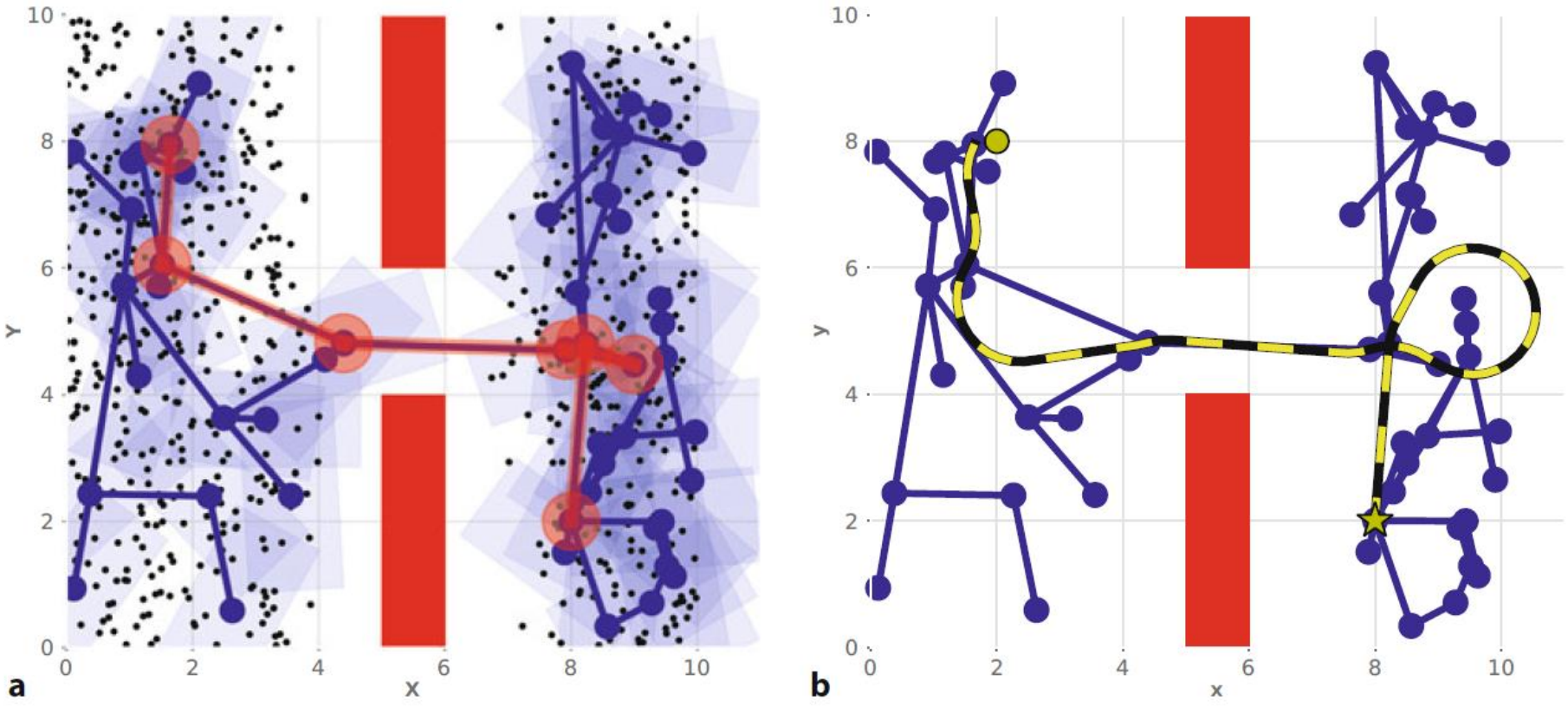
* Close in terms of suitable combination of distance and orientation

Rapidly-Exploring Random Tree (RRT)

In Python:

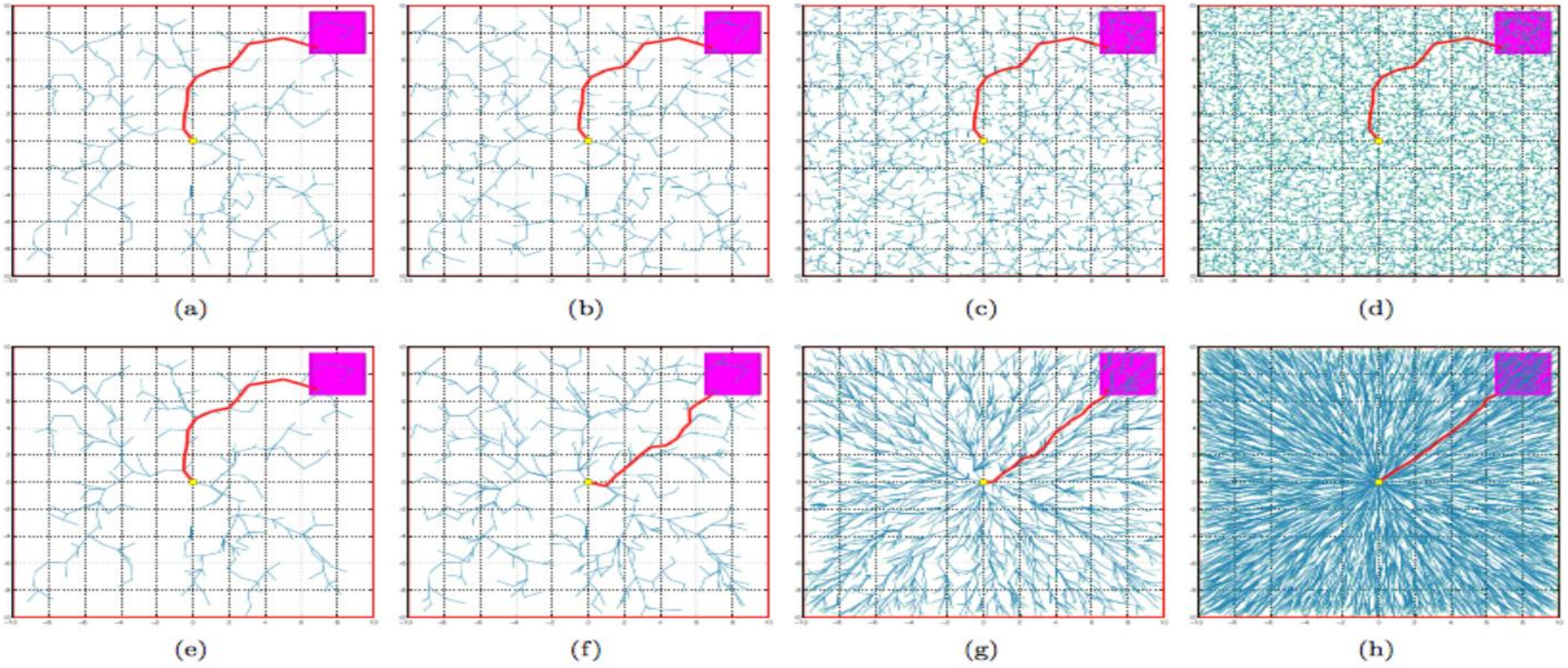
```
>>> vehicle = Bicycle(steer_max=1, L=2, polygon=vpolygon);
>>> rrt = RRTPlanner(map=map, vehicle=vehicle, npoints=50,showsamples=True, seed=0)
>>> rrt.plan(goal=qg, animate=True)
>>> path, status = rrt.query(start=qs);
>>> rrt.g.plot()
>>> rrt.g.highlight_path(status.vertices, color="red")
>>> rrt.plot(path);
```

Rapidly-Exploring Random Tree (RRT)



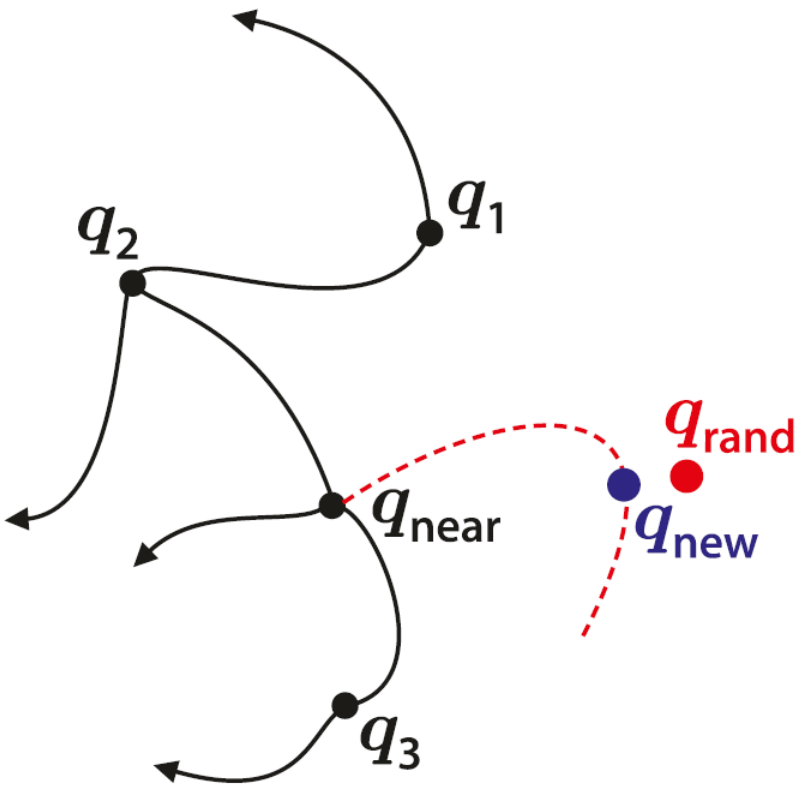
RRT vs. RRT*

- RRT finds **one** solution with probability -> 1 (no quality/optimality guaranty)
- RRT* finds **optimal** solution with probability -> 1 (Karaman&Frazzoli,2011)



RRT vs. RRT*

1. Generate a new random config, q_{rand}
2. Find closest* config, q_{near}
3. Compute feasible control that moves robot from q_{near} towards q_{rand}
4. If no collision, ~~add closest config q_{new} to graph. Goto1~~
   a) Find set of Neighbors, N of closest config q_{new}
   b) Choose best parent and adopt new children (if improvement). Goto1



Planning algorithms covered:

Reactive

- **Braitenberg vehicle:** purely reactive, non-complete, non-optimal
- **Bug0, Bug1, Bug2:** increasingly complex/capable: added limited memory, state-machine based automata, non-complete, non-optimal

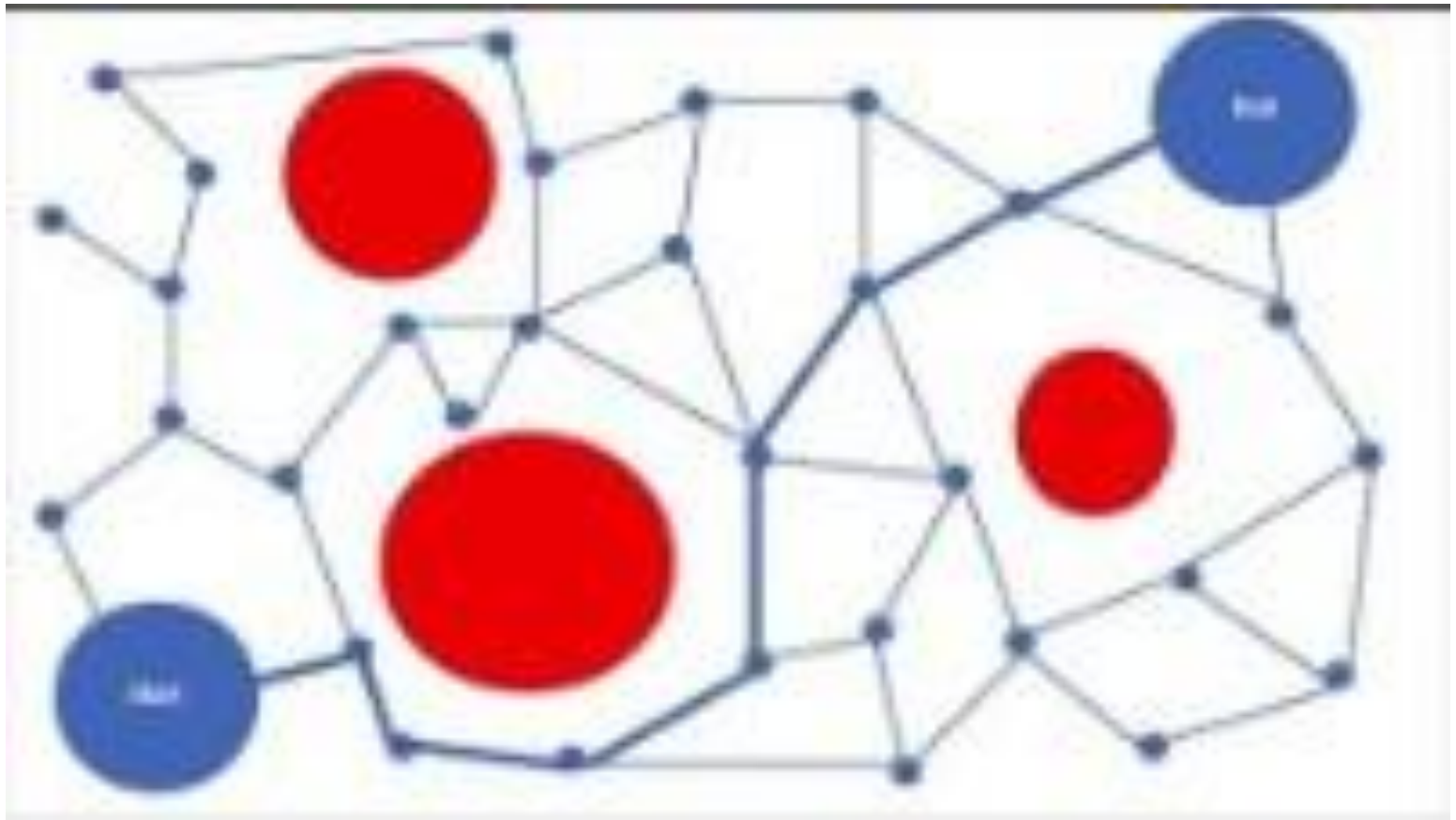
Planning algorithms covered:

Map-based

- **Distance transform:** computationally expensive, optimal, independent of start, dependent on goal configuration
- **D*:** optimal, generalized cost, cheap incremental re-planning, , independent of start, dependent on goal configuration
- **PRM:** sample- and roadmap based, computationally efficient, non-optimal, independent of start/goal configuration, struggles with narrow passages
- **Dubins'- and Reed-Shepp's:** time-optimal, continuous, but non-differentiable paths
- **Latice planner:** roadmap based, kino-dynamically feasible paths
- **RRT/RRT*:** sample- and roadmap based, kino-dynamically feasible paths

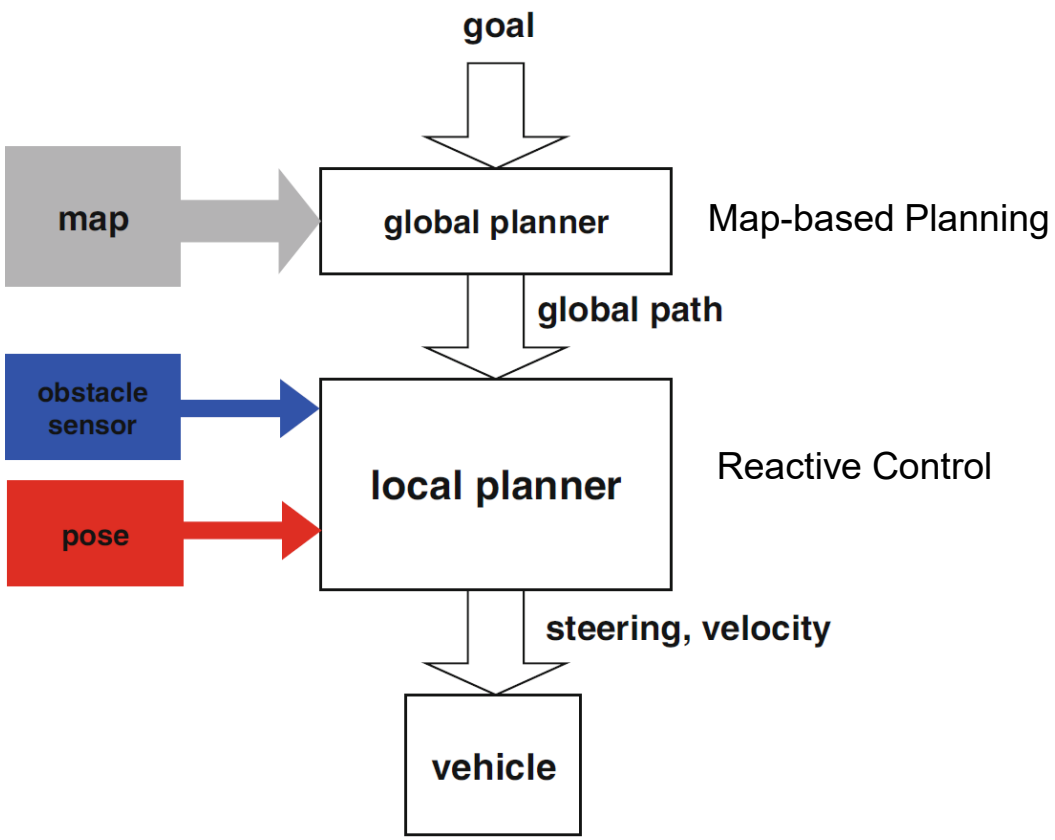
All map-based methods require localization and mapping capabilities (SLAM)!

Motion Planning Algorithms (PRM, RRT, RRT*)



Local and Global Planning

- Assuming the map is correct, global planner provides optimal paths
- BUT: “reality-gap” necessitates for reactive, local planning and control
  - Using current state (pose) and sensor readings
- In practice, we use both map-based planning and reactive strategies.



“Homework”: See Sertac Karaman (MIT) on Motion Planning in a Complex World (https://youtu.be/0fLSf3NO0-s)

Thank you for your attention!